

Interpretability

ACTL3143 & ACTL5111 Deep Learning for Actuaries
Patrick Laub

Lecture Outline

- **Introduction**
- Illustrative Example
- Inherently Interpretable Models
- Post-hoc Explanations
- Belgian Motor Dataset
- Interpreting Inherently Interpretable Models
- Neural Additive Models on the Belgian Data
- Explaining a Neural Network with Partial Dependence Plots
- Explaining Specific Models

Definitions

“Surprisingly enough, although the concept of interpretability is increasingly widespread, there is no general consensus on both the definition and the measurement of the interpretability of a model.” (Delcaillau et al., 2022)

“Definition Interpret means *to explain or to present in understandable terms*. In the context of ML systems, we define interpretability as *the ability to explain or to present in understandable terms to a human*.” (Doshi-Velez & Kim, 2017)

A distinction between interpretability and explainability

“Interpretability is about transparency, about understanding exactly why and how the model is generating predictions, and therefore, it is important to observe the inner mechanics of the algorithm considered. This leads to interpreting the model’s parameters and features used to determine the given output. Explainability is about explaining the behavior of the model in human terms.” (Charpentier, 2024)

Aspects of Interpretability

Inherent Interpretability

The model is interpretable by design.

Post-hoc Explanations

The model is not interpretable by design, but we can use other methods to explain the model.

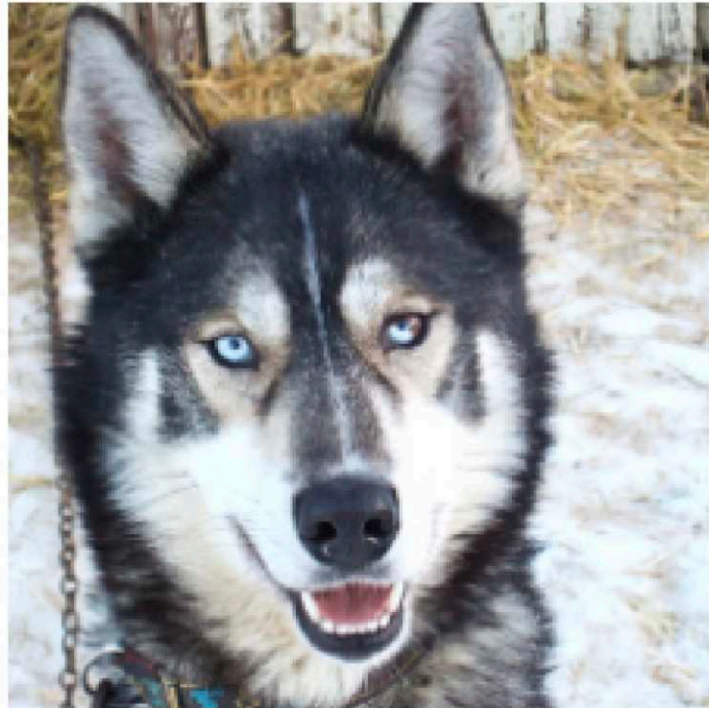
Global Interpretability

The ability to understand how the model works.

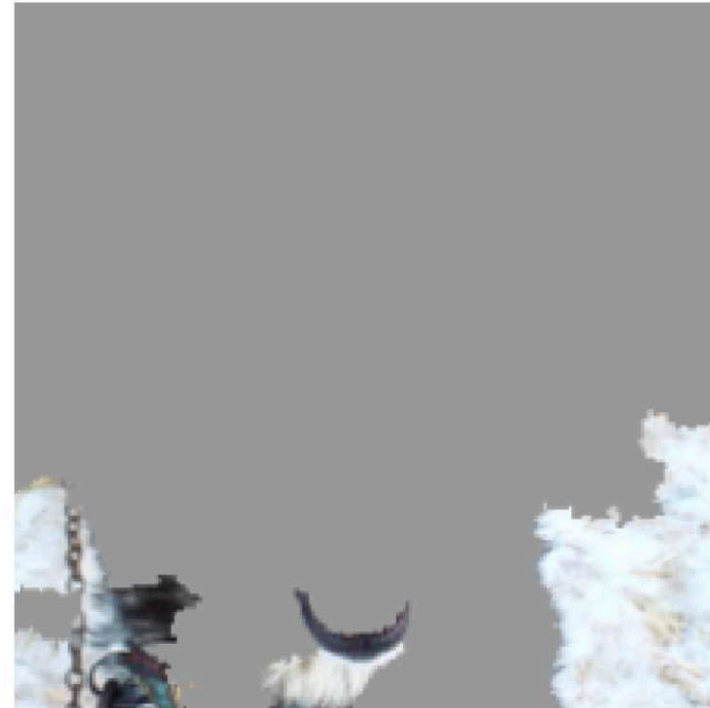
Local Interpretability

The ability to interpret/understand each prediction.

Husky vs. Wolf



(a) Husky classified as wolf



(b) Explanation

Figure 11: Raw data and explanation of a bad model's prediction in the "Husky vs Wolf" task.

A well-known anecdote in the explainability literature ([Ribeiro et al., 2016](#)).

Adversarial examples



$+ .007 \times$



$=$



x

“panda”

57.7% confidence

$\text{sign}(\nabla_x J(\theta, x, y))$

“nematode”

8.2% confidence

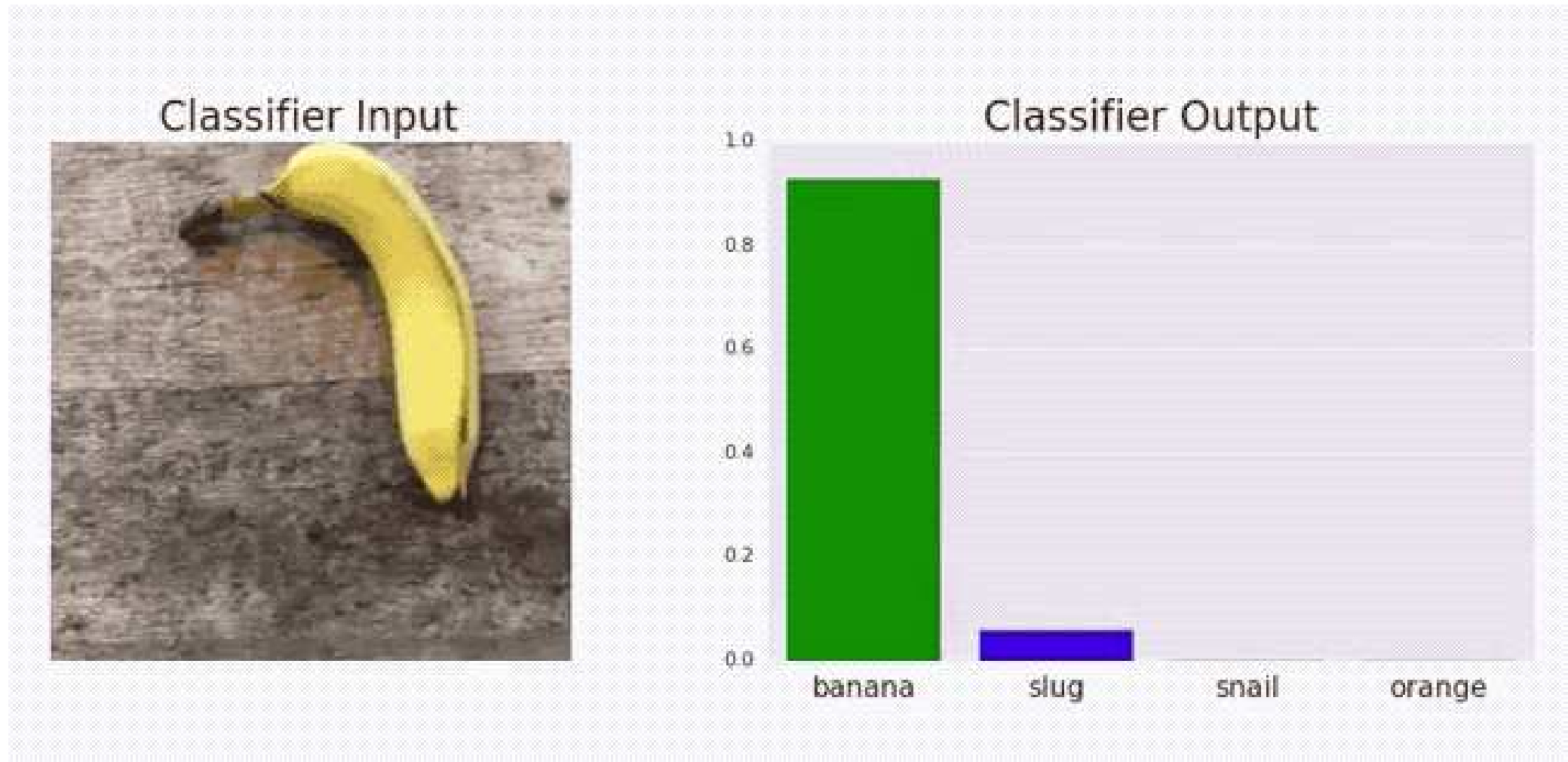
$x + \epsilon \text{sign}(\nabla_x J(\theta, x, y))$

“gibbon”

99.3 % confidence

A demonstration of fast adversarial example generation applied to GoogLeNet on ImageNet. By adding an imperceptibly small vector whose elements are equal to the sign of the elements of the gradient of the cost function with respect to the input, we can change GoogLeNet’s classification of the image (Goodfellow et al., 2014)

Adversarial stickers

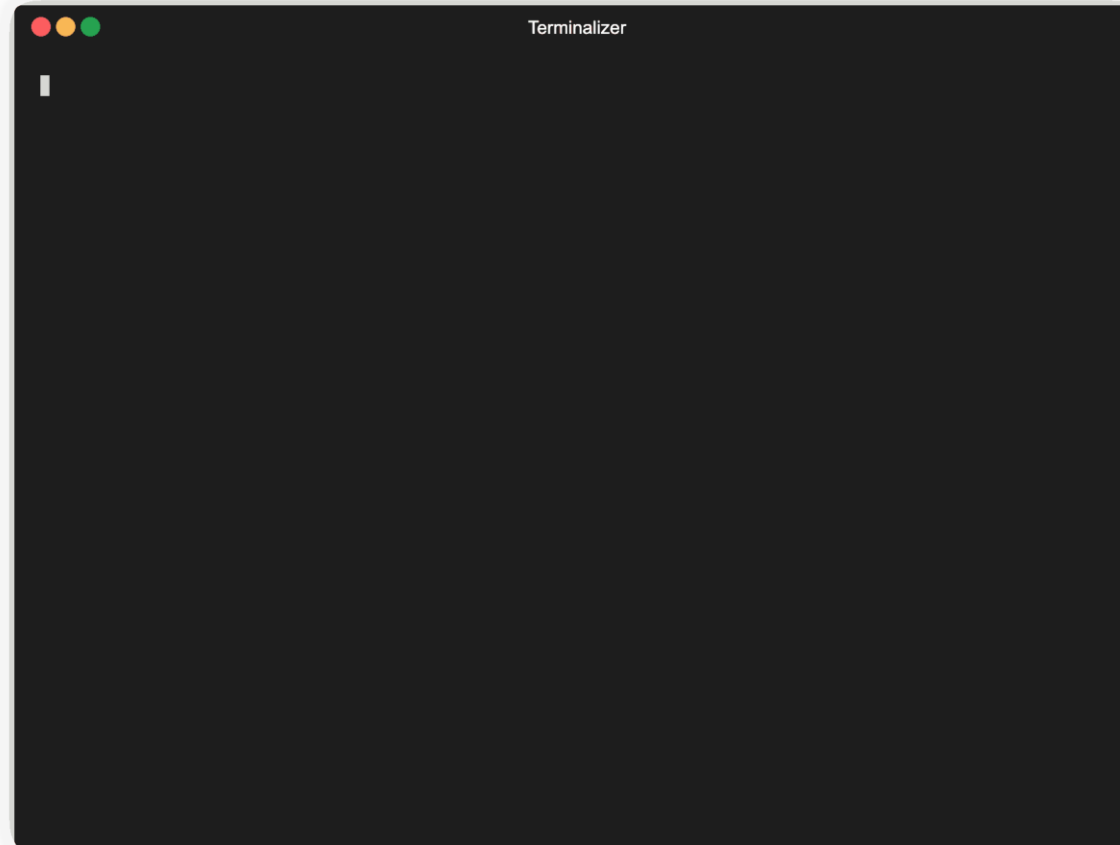


Adversarial stickers.

Source: The Verge (2018), [These stickers make computer vision software hallucinate things that aren't there.](#)

Adversarial text

“TextAttack 🐙 is a Python framework for adversarial attacks, data augmentation, and model training in NLP”



Demo

Lecture Outline

- Introduction
- **Illustrative Example**
- Inherently Interpretable Models
- Post-hoc Explanations
- Belgian Motor Dataset
- Interpreting Inherently Interpretable Models
- Neural Additive Models on the Belgian Data
- Explaining a Neural Network with Partial Dependence Plots
- Explaining Specific Models

Package imports

```
1 import random
2
3 import matplotlib.pyplot as plt
4 import numpy as np
5 import numpy.random as rnd
6 import pandas as pd
7
8 import geopandas as gpd
9 import contextily as ctx
10 from shapely.geometry import Point
11 import plotly.graph_objects as go
12
13 import keras
14 from keras.metrics import SparseTopKCategoricalAccuracy
15 from keras.models import Sequential, Model
16 from keras.layers import Dense, Input
17 from keras.callbacks import EarlyStopping
18
19 from sklearn import tree, set_config
20 from sklearn.preprocessing import LabelEncoder, OrdinalEncoder, StandardScaler, OneHotEncoder
21 from sklearn.feature_extraction.text import CountVectorizer
22 from sklearn.model_selection import train_test_split
23 from sklearn.compose import make_column_transformer
24 from sklearn.metrics import mean_poisson_deviance, mean_absolute_error
```

First attempt at NLP task

```
1 df_raw["SUMMARY_EN"]
```

```
0      V1, a 2000 Pontiac Montana minivan,
made a left turn...
1      The crash occurred in the eastbound
lane of a ...
2      This crash occurred just after the
noon time hour...

...
6946    The crash occurred in the eastbound
lanes of a ...
6947    This single-vehicle crash occurred in
a rural ...
6948    This two vehicle daytime collision
occurred miles ...
Name: SUMMARY_EN, Length: 6949, dtype: str
```

```
1 df_raw["NUM_VEHICLES"].value_counts()\
2     .sort_index()
```

```
NUM_VEHICLES
1      1822
2      4151
3+      976
Name: count, dtype: int64
```

Bag of words for the top 1,000 words

```

1 vect = CountVectorizer(max_features=1_000, stop_words="english")
2 vect.fit(X_train["SUMMARY_EN"])
3
4 X_train_bow = vectorise_dataset(X_train, vect)
5 X_val_bow = vectorise_dataset(X_val, vect)
6 X_test_bow = vectorise_dataset(X_test, vect)
7
8 vectorise_dataset(X_train, vect, dataframe=True).head()

```

	10	105	113	12	15	150	16	17	18	180	...	yield	zone	WEATHER1	V
2532	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
6209	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
2561	1	0	1	0	0	0	0	0	0	0	...	0	0	0	0
6664	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
4214	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0

5 rows × 1008 columns

Trained a basic neural network on that

```

1 num_features = X_train_bow.shape[1]
2 num_cats = df_raw["NUM_VEHICLES"].nunique()
3 model = build_model(num_features, num_cats)
4 es = EarlyStopping(patience=1, restore_best_weights=True, monitor="val_accuracy")
5 model.fit(X_train_bow, y_train, epochs=10,
6         callbacks=[es], validation_data=(X_val_bow, y_val), verbose=0)
7 model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 100)	100,900
dense_1 (Dense)	(None, 3)	303

Total params: 303,611 (1.16 MB)

Trainable params: 101,203 (395.32 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 202,408 (790.66 KB)

```
1 model.evaluate(X_train_bow, y_train, verbose=0)
```

```
[0.03331480547785759, 0.9949628114700317, 0.9997601509094238]
```

```
1 model.evaluate(X_val_bow, y_val, verbose=0)
```

```
[0.09635408222675323, 0.9748201370239258, 0.9978417158126831]
```

Permutation importance algorithm

Taken directly from scikit-learn documentation:

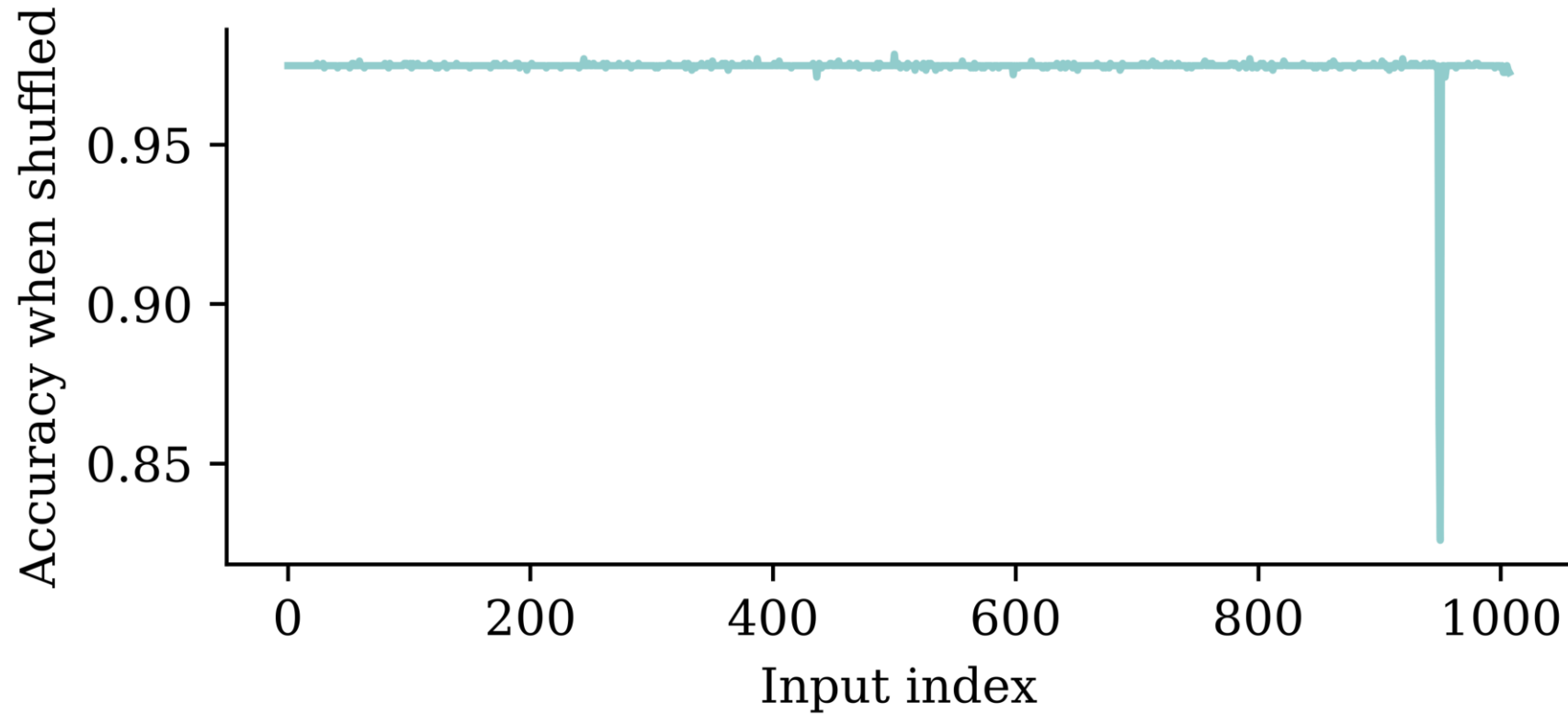
- Inputs: fitted predictive model m , tabular dataset (training or validation) D .
- Compute the reference score s of the model m on data D (for instance the accuracy for a classifier or the R^2 for a regressor).
- For each feature j (column of D):
 - For each repetition k in $1, \dots, K$:
 - Randomly shuffle column j of dataset D to generate a corrupted version of the data named $\tilde{D}_{k,j}$.
 - Compute the score $s_{k,j}$ of model m on corrupted data $\tilde{D}_{k,j}$.
 - Compute importance i_j for feature f_j defined as:

$$i_j = s - \frac{1}{K} \sum_{k=1}^K s_{k,j}$$

Source: scikit-learn documentation, [permutation_importance function](#).

Run the permutation test

```
1 all_perm_scores = permutation_test(model, X_val_bow, y_val)
```



Find the most significant inputs

```
['v3', 'v2', 'vehicle', 'harmful', 'v4', 'motor', 'WEATHER8', 'WEATHER5', 'WEATHER4',
'WEATHER3', 'posted', 'trailer', 'parked', 'drove', 'chevrolet', 'event', 'single',
'WEATHER7', 'lane', 'light', 'legally', 'steered', 'traffic', 'continued', 'rest', 'concrete',
'afternoon', 'surface', 'arterial', 'asphalt', 'coded', 'second', 'seizure', 'seven',
'shortly', 'change', 'sightline', 'sign', 'struck', 'signs', 'striking', 'braking', 'began',
'spun', 'started', 'steady', 'attack', 'researcher', 'directions', 'decision', 'medication',
'median', 'maneuver', 'male', 'make', 'gas', 'meters', 'located', 'heading', 'leaving',
'kilometers', 'involved', 'inadequate', 'information', 'limit', 'crossed', 'minivan', 'mph',
'pre', 'pole', 'plane', 'direction', 'pickup', 'drivers', 'movement', 'pain', 'ordered',
'oncoming', 'encroaching', 'observed', 'number', 'noticed', 'dry', 'travel', 'injuries',
'2006', 'waiting', 'year', 'vehicles', 'utility', '30', 'treatment', 'WEATHER1', '38', '48',
'ability', 'notice', 'noted', 'numerous', 'object']
```

How about a simple decision tree?

```
1 clf = tree.DecisionTreeClassifier(random_state=0, max_leaf_nodes=3)
2 clf.fit(X_train_bow[:, best_input_inds], y_train);
```

```
1 print(clf.score(X_train_bow[:, best_input_inds], y_train))
2 print(clf.score(X_val_bow[:, best_input_inds], y_val))
```

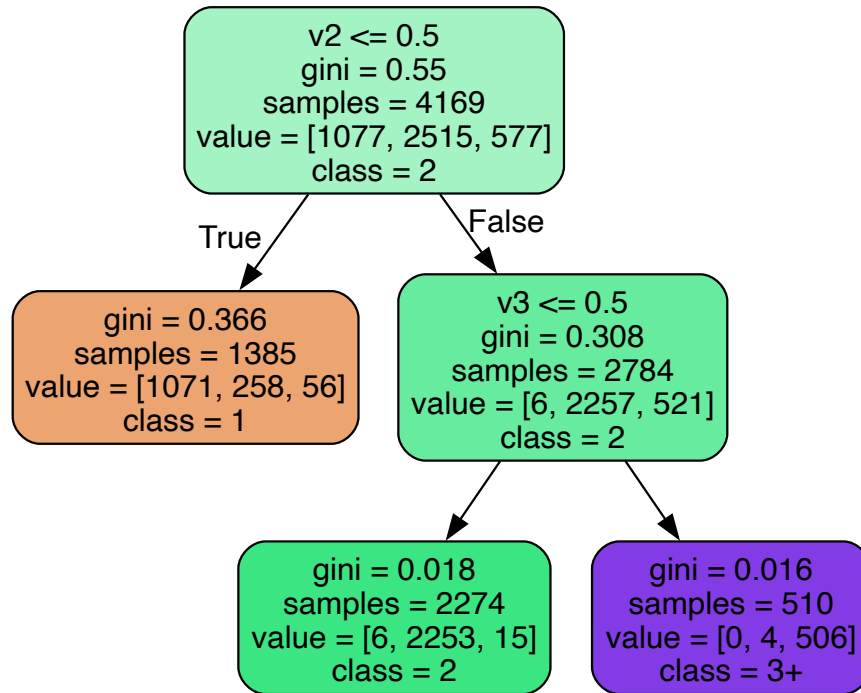
0.9186855360997841

0.9316546762589928

The decision tree ends up giving pretty good results.

Decision tree

```
1 tree.plot_tree(clf, feature_names=best_inputs, filled=True);
```



```
1 print(np.where(clf.feature_importances_ > 0)[0])
2 [best_inputs[ind] for ind in np.where(clf.feature_importances_ > 0)[0]]
```

```
[0 1]
```

```
['v3', 'v2']
```

This is why we replace “v1”, “v2”, “v3”

There was a slide in the NLP deck titled “Just ignore this for now...” That was going through each summary and replacing the words “V1”, “V2”, “V3” with random numbers. This was done to see if the model was overfitting to these words.

```
1 model.fit(X_train_bow, y_train, epochs=10,  
2         callbacks=[es], validation_data=(X_val_bow, y_val), verbose=0);
```

Retraining on the fixed dataset gives us a more realistic (lower) accuracy.

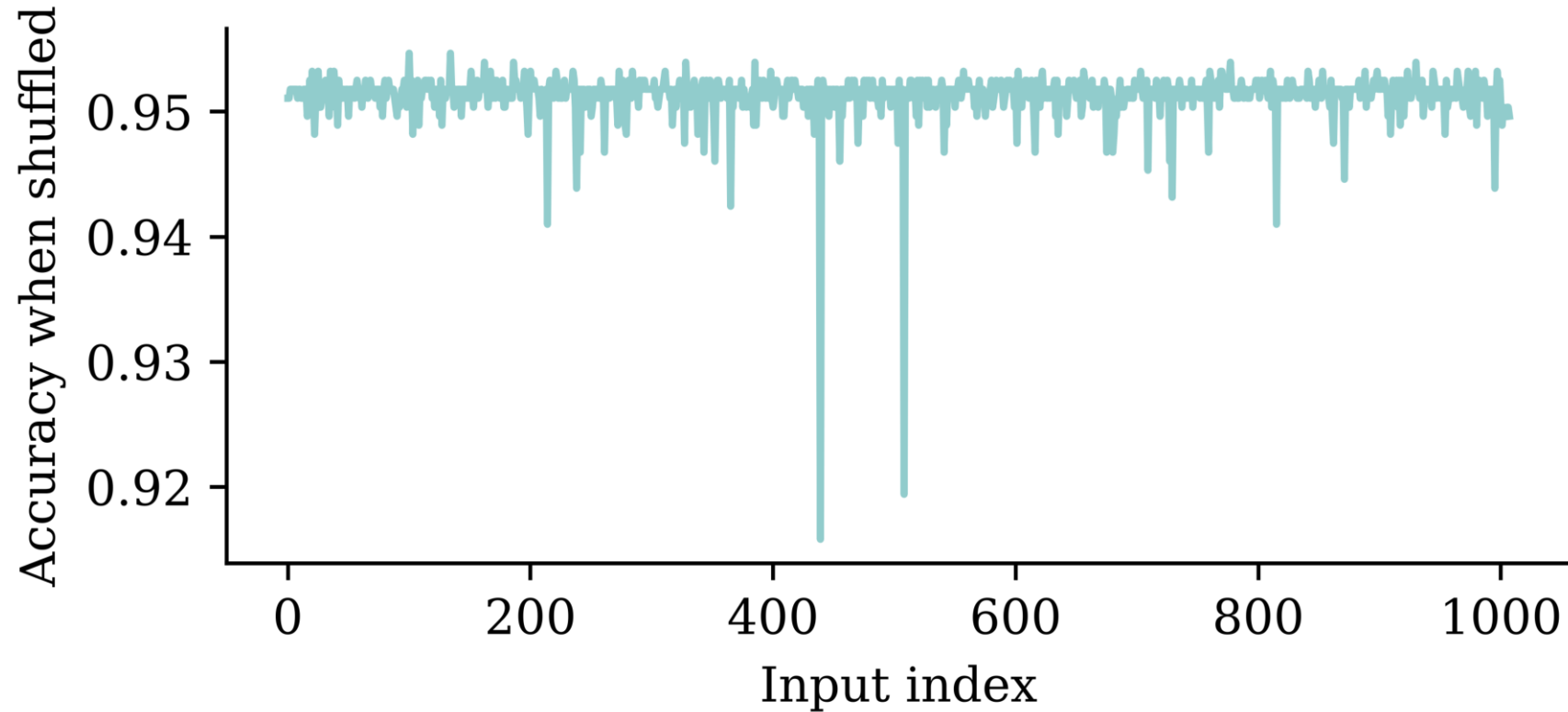
```
1 model.evaluate(X_train_bow, y_train, verbose=0)
```

```
[0.022723009809851646, 0.9990405440330505, 1.0]
```

```
1 model.evaluate(X_val_bow, y_val, verbose=0)
```

```
[0.16461113095283508, 0.9517985582351685, 0.9964028596878052]
```

Permutation importance accuracy plot



Find the most significant inputs

```

1 vocab = vect.get_feature_names_out()
2 input_cols = list(vocab) + weather_cols
3
4 best_input_inds = np.argsort(perm_scores)[:100]
5 best_inputs = [input_cols[idx] for idx in best_input_inds]
6
7 print(best_inputs)

```

```

['harmful', 'involved', 'single', 'coded', 'event', 'reason', 'year', 'contacted', 'struck',
'pushed', 'hit', 'encroachment', 'rear', 'continued', 'road', 'pickup', 'non', 'pole',
'limit', 'edge', 'critical', 'driven', 'impact', 'motor', 'intersection', 'stopped', 'police',
'guardrail', 'dry', 'vehicles', 'chevrolet', 'daylight', 'afternoon', '1999', 'occurred',
'traffic', 'day', 'alcohol', 'failure', 'WEATHER2', 'associated', 'westbound', 'familiar',
'interview', '30', 'lane', 'line', 'dodge', 'traveling', 'daily', 'away', 'encroaching',
'corner', 'asphalt', 'clear', 'ford', 'kph', 'grand', 'WEATHER5', 'WEATHER4', 'work',
'weekday', 'undivided', 'treated', 'toyota', 'stop', 'ran', 'poor', 'forward', 'parked',
'occupants', 'nissan', 'median', 'male', 'interviewed', 'injured', 'inadequate', 'impairment',
'home', 'heard', 'opposite', 'according', 'WEATHER8', '37', '1993', '23', '72', 'drives',
'right', 'contacting', '2003', 'contributed', 'unsuccessful', 'performance', 'paved',
'behavior', 'roof', 'brake', '46', 'initial']

```

Lecture Outline

- Introduction
- Illustrative Example
- **Inherently Interpretable Models**
- Post-hoc Explanations
- Belgian Motor Dataset
- Interpreting Inherently Interpretable Models
- Neural Additive Models on the Belgian Data
- Explaining a Neural Network with Partial Dependence Plots
- Explaining Specific Models

What is inherent interpretability?

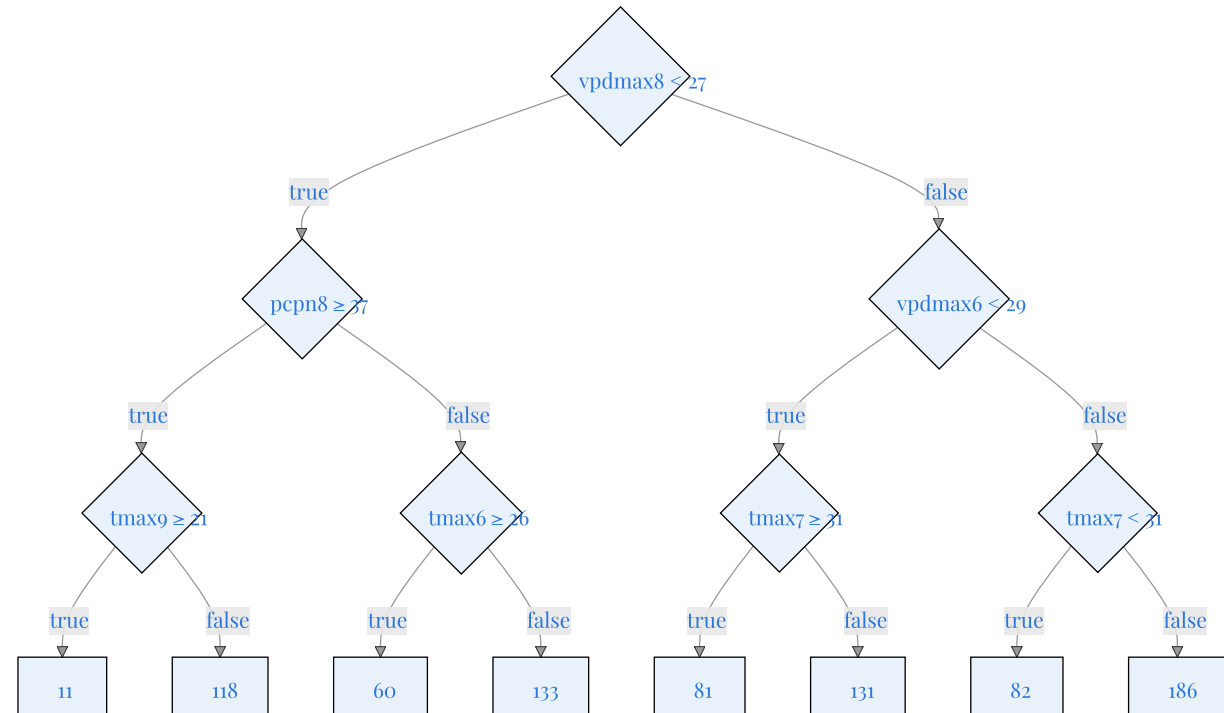
“Interpretability by design is decided on the level of the machine learning algorithm. If you want a machine learning algorithm that produces interpretable models, the algorithm has to constrain the search of models to those that are interpretable. The simplest example is linear regression: When you use ordinary least squares to fit/train a linear regression model, you are using an algorithm that will produce ... models that are linear in the input features. Models that are interpretable by design are also called **intrinsically** or **inherently** interpretable models.” (Molnar, 2020)



Christoph Molnar

Examples of interpretable models

- Linear regression
- Generalised linear models
- Decision trees
- Decision rules



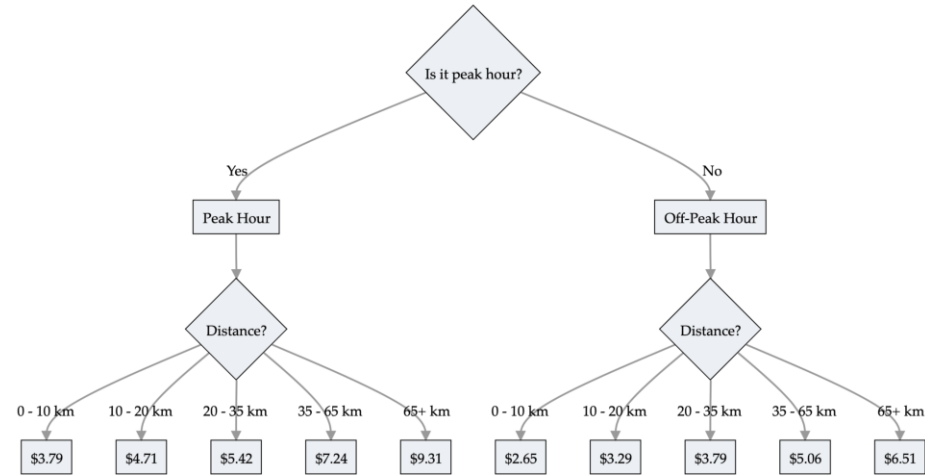
E.g. decision tree for payouts of index insurance
(Chen et al., 2024).

Better trees

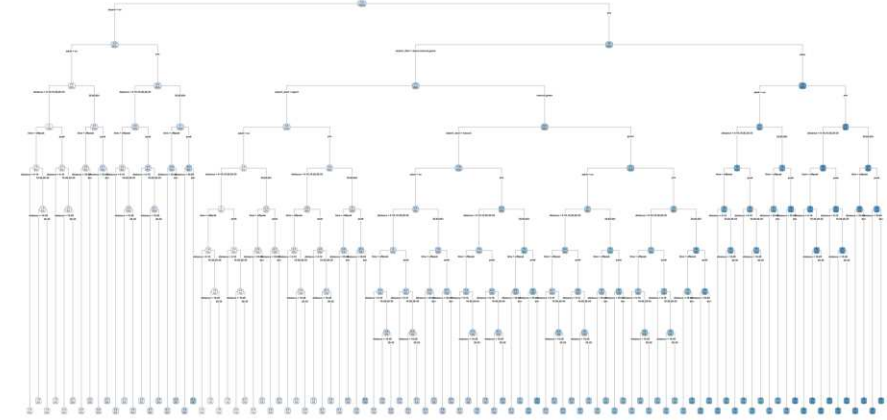
“The optimization over the node parameters (exact for axis-aligned trees, approximate for oblique trees) assumes the rest of the tree (structure and parameters) is fixed. The greedy nature of the algorithm means that once a node is optimized, it its [sic] fixed forever.” (Carreira-Perpinán & Tavallali, 2018)

Non-greedy search can improve the accuracy of the tree, without sacrificing interpretability.

Some processes don't fit the tree structure



Train prices



Full train pricing

Decision rules

Make predictions using if-then statements related to the inputs.

“Decision rules can be **as expressive as decision trees**, while being more **compact**. Decision trees often also suffer from replicated sub-trees, that is, when the splits in a left and a right child node have the same structure.”
(Molnar, 2020)

```

1  def rail_cost(peak_hours, distance):
2      if peak_hours:
3          if distance ≤ 10:
4              cost = 3.79
5          elif distance ≤ 20:
6              cost = 4.71
7          elif distance ≤ 35:
8              cost = 5.42
9          elif distance ≤ 65:
10             cost = 7.24
11         else:
12             cost = 9.31
13     else:
14         if distance ≤ 10:
15             cost = 2.65
16         elif distance ≤ 20:
17             cost = 3.29
18         elif distance ≤ 35:
19             cost = 3.79
20         elif distance ≤ 65:
21             cost = 5.06
22         else:
23             cost = 6.51
24     return cost

```

Scoring rules

1.	Prior arrests ≥ 2	1 point	...
2.	Prior arrests ≥ 5	1 point	+ ...
3.	Prior arrests for local ordinance	1 point	+ ...
4.	Age at Release between 18 to 24	1 point	+ ...
5.	Age at Release ≥ 40	-1 point	+ ...
SCORE			= ...

Score	-1	0	1	2	3	4
Risk (%)	11.9	26.9	50.0	73.1	88.1	95.3

When to choose inherent interpretability?

Perspective | [Published: 13 May 2019](#)

Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead

[Cynthia Rudin](#) 

[Nature Machine Intelligence](#) 1, 206–215 (2019) | [Cite this article](#)

66k Accesses | 2230 Citations | 485 Altmetric | [Metrics](#)

Abstract

Black box machine learning models are currently being used for high-stakes decision making throughout society, causing problems in healthcare, criminal justice and other domains. Some people hope that creating methods for explaining these black box models will alleviate some of the problems, but trying to explain black box models, rather than creating models that are interpretable in the first place, is likely to perpetuate bad practice and can potentially cause great harm to society. The way forward is to design models that are inherently interpretable. This Perspective clarifies the chasm between explaining black boxes and using inherently interpretable models, outlines several key reasons why explainable black boxes should be avoided in high-stakes decisions, identifies challenges to interpretable machine learning, and provides several example applications where interpretable models could potentially replace black box models in criminal justice, healthcare and computer vision.



Cynthia Rudin

Rudin (2019)

i Article 22 GDPR – Automated individual decision-making, including profiling

1. The data subject shall have the right **not** to be subject to a decision based solely on automated processing, including profiling, which produces legal effects concerning him or her or similarly significantly affects him or her.
2. Paragraph 1 shall not apply if the decision:
 - a. is necessary for entering into, or performance of, a contract between the data subject and a data controller;
 - b. is authorised by Union or Member State law to which the controller is subject and which also lays down suitable measures to safeguard the data subject's rights and freedoms and legitimate interests; or
 - c. is based on the data subject's explicit consent.
3. In the cases referred to in points (a) and (c) of paragraph 2, the data controller shall implement suitable measures to safeguard the data subject's rights and freedoms and legitimate interests, at least the right to obtain human intervention on the part of the controller, to express his or her point of view and to contest the decision.
4. Decisions referred to in paragraph 2 shall not be based on special categories of personal data referred to in Article 9(1), unless point (a) or (g) of Article 9(2) applies and suitable measures to safeguard the data subject's rights and freedoms and legitimate interests are in place.

Linear models & LocalGLMNet

A GLM has the form

$$\hat{y} = g^{-1}(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p)$$

where $\beta_0, \dots, \beta_p$ are the model parameters.

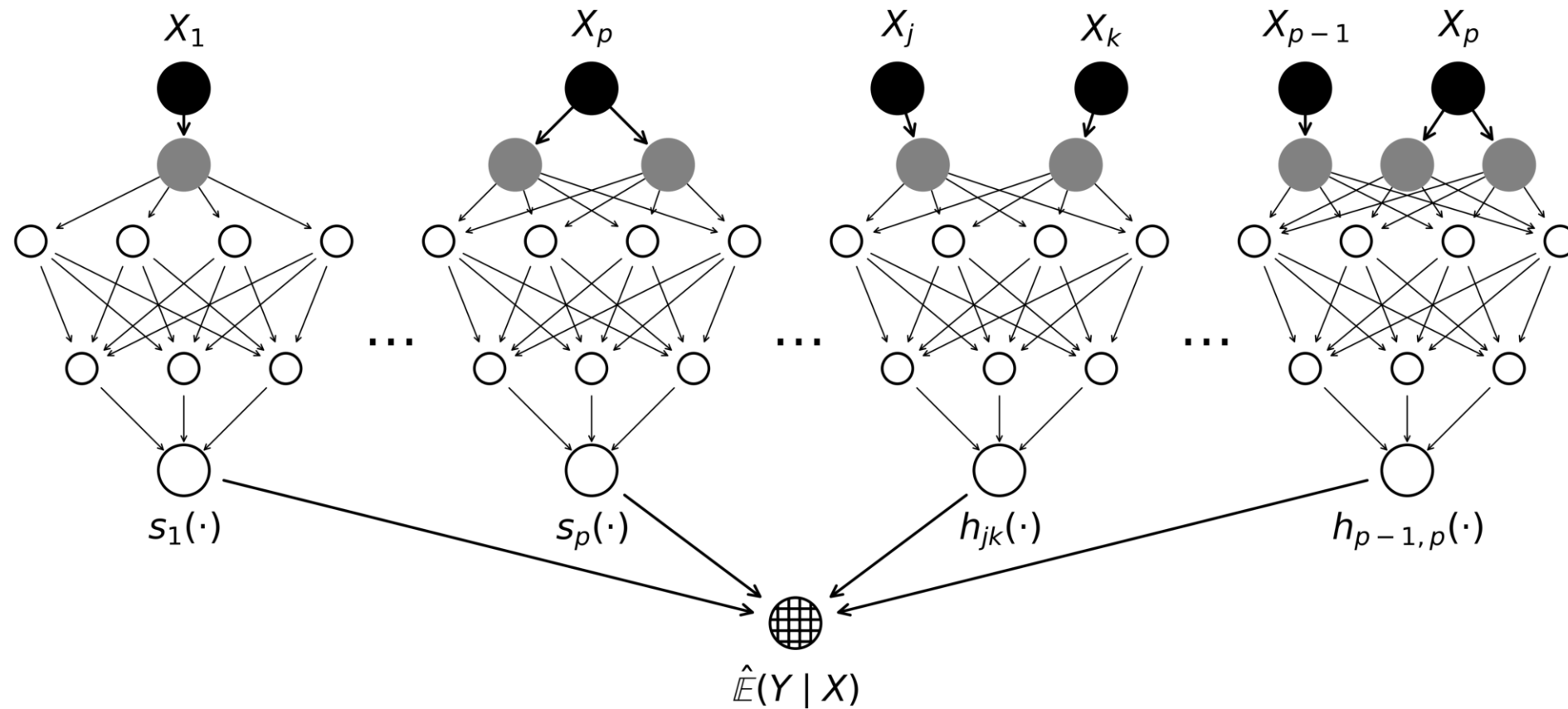
Global & local interpretations are easy to obtain.

LocalGLMNet extends this to a neural network (Richman & Wüthrich, 2023).

$$\hat{y}_i = g^{-1}(\beta_0(\mathbf{x}_i) + \beta_1(\mathbf{x}_i)x_{i1} + \cdots + \beta_p(\mathbf{x}_i)x_{ip})$$

A GLM with local parameters $\beta_0(\mathbf{x}_i), \dots, \beta_p(\mathbf{x}_i)$ for each observation $\mathbf{x}_i$. The local parameters are the output of a neural network.

Neural Additive Models (NAMs)



Each covariate (or select interactions) receives its own subnetwork, which contributes additively.

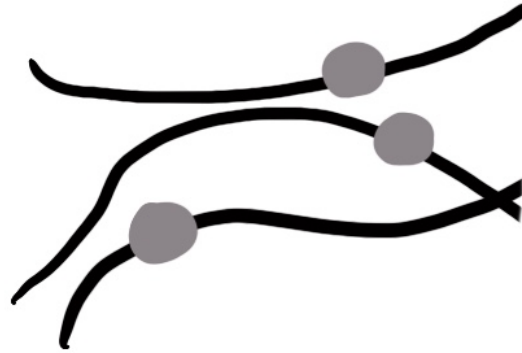
Lecture Outline

- Introduction
- Illustrative Example
- Inherently Interpretable Models
- **Post-hoc Explanations**
- Belgian Motor Dataset
- Interpreting Inherently Interpretable Models
- Neural Additive Models on the Belgian Data
- Explaining a Neural Network with Partial Dependence Plots
- Explaining Specific Models

One variable's effect on the prediction



CETERIS PARIBUS



ICE



PDP

Various plots showing the effect of one variable on the prediction; from ([Molnar, 2020, sec. 12](#))

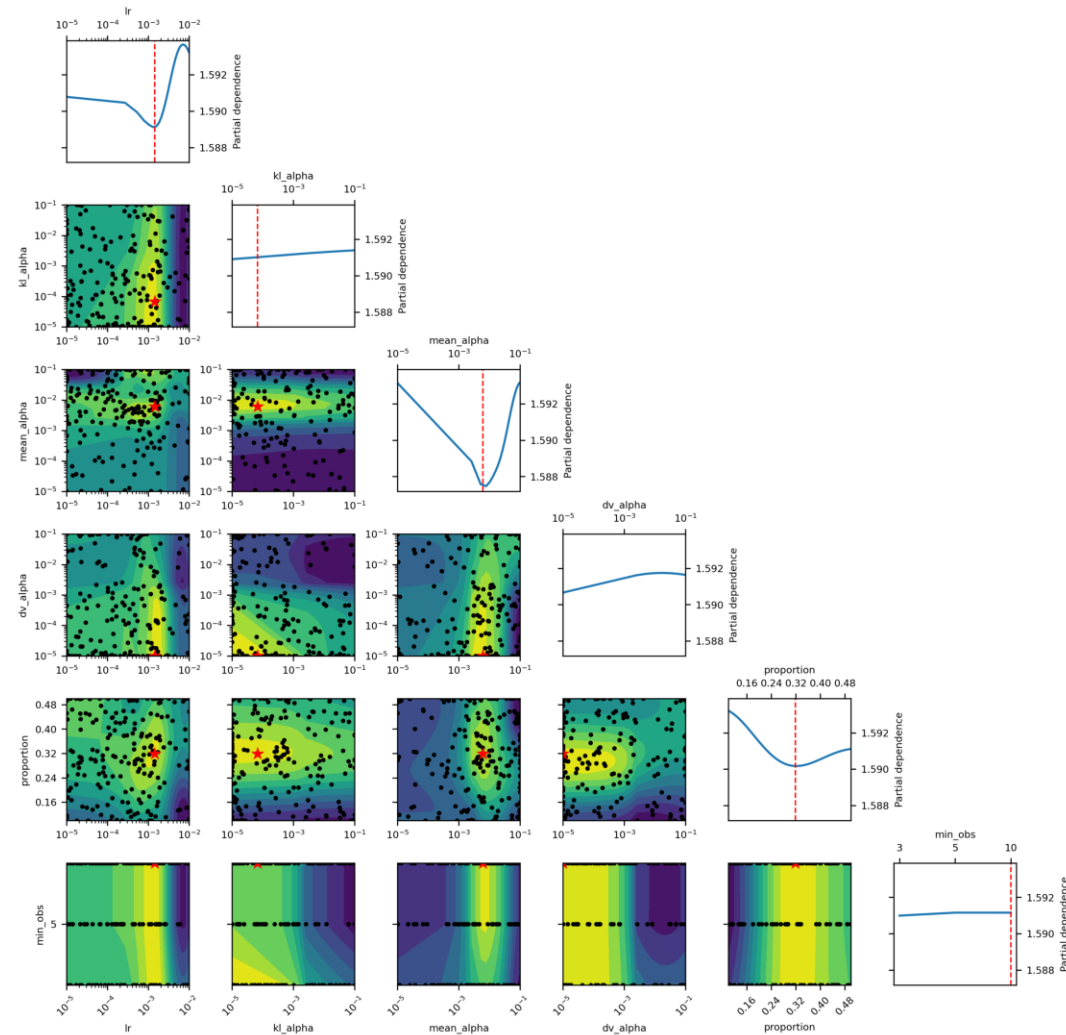
The different plots

A *Ceteris Paribus* plot shows how the model's prediction changes as we vary the value of *one covariate/input* while keeping *all other features constant* at their original values for some observation.

An *Individual Conditional Expectation* (ICE) plot is the same thing as a ceteris paribus plot, but for *all observations* in the dataset.

A *Partial Dependence Plot* (PDP) shows how the model's prediction changes as we vary the value of *one covariate/input* while averaging over all other features in the dataset.

Useful for Hyperparameter Tuning



Partial dependence plots for hyperparameter tuning

Permutation importance

- Inputs: fitted model m , tabular dataset D .
- Compute the reference score s of the model m on data D (for instance the accuracy for a classifier or the R^2 for a regressor).
- For each feature j (column of D):
 - For each repetition k in $1, \dots, K$:
 - Randomly shuffle column j of dataset D to generate a corrupted version of the data named $\tilde{D}_{k,j}$.
 - Compute the score $s_{k,j}$ of model m on corrupted data $\tilde{D}_{k,j}$.
 - Compute importance i_j for feature f_j defined as:

$$i_j = s - \frac{1}{K} \sum_{k=1}^K s_{k,j}$$

Originally proposed by Breiman (2001), extended by Fisher et al. (2019).

Source: scikit-learn documentation, [permutation_importance](#) function.

Permutation importance

```

1  def permutation_test(model, X, y, num_reps=1, seed=42, batch_size=8192):
2      """
3      Run the permutation test for variable importance.
4      Returns matrix of shape (X.shape[1], len(model.evaluate(X, y))).
5      """
6      rnd.seed(seed)
7      scores = []
8
9      for j in range(X.shape[1]):
10         original_column = np.copy(X[:, j])
11         col_scores = []
12
13         for r in range(num_reps):
14             rnd.shuffle(X[:, j])
15             col_scores.append(model.evaluate(X, y, verbose=0, batch_size=batch_size))
16
17         scores.append(np.mean(col_scores, axis=0))
18         X[:, j] = original_column
19
20     return np.array(scores)

```


LIME

Local Interpretable Model-agnostic Explanations employs an interpretable surrogate model to explain locally how the black-box model makes predictions for individual instances.

E.g. a black-box model predicts Bob's premium as the highest among all policyholders. LIME uses an interpretable model (a linear regression) to explain how Bob's features influence the black-box model's prediction.

See “Why Should I Trust You?": Explaining the Predictions of Any Classifier.

LIME Algorithm

Suppose we want to explain the instance $\mathbf{x}_{\text{Bob}} = (1, 2, 0.5)$.

1. Generate perturbed examples of $\mathbf{x}_{\text{Bob}}$ and use the trained gamma MDN f to make predictions:

$$\begin{aligned} \mathbf{x}_{\text{Bob}}^{(1)} &= (1.1, 1.9, 0.6), & f(\mathbf{x}_{\text{Bob}}^{(1)}) &= 34000 \\ \mathbf{x}_{\text{Bob}}^{(2)} &= (0.8, 2.1, 0.4), & f(\mathbf{x}_{\text{Bob}}^{(2)}) &= 31000 \\ &\vdots & &\vdots \end{aligned}$$

We can then construct a dataset of N_{Examples} perturbed examples: $\mathcal{D}_{\text{LIME}} = (\{\mathbf{x}_{\text{Bob}}^{(i)}, f(\mathbf{x}_{\text{Bob}}^{(i)})\})_{i=0}^{N_{\text{Examples}}}$.

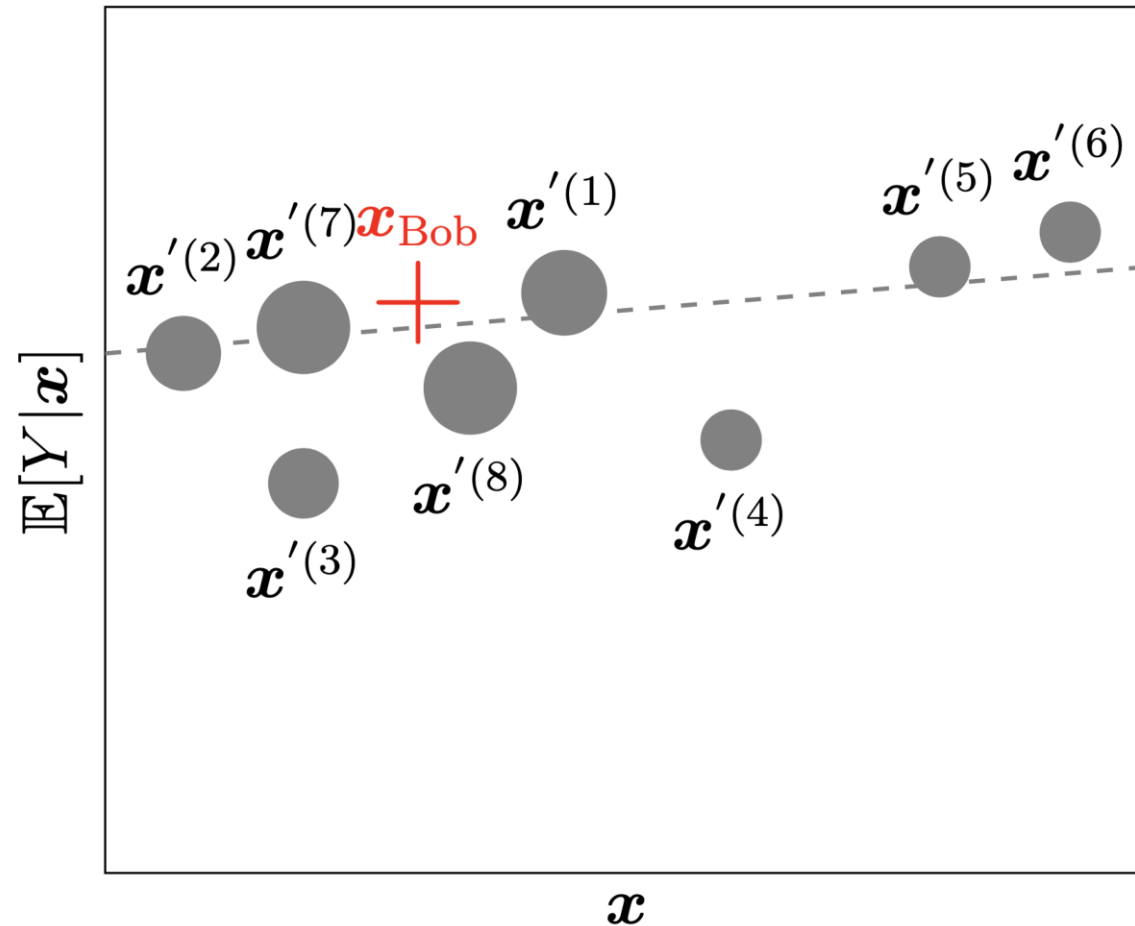
LIME Algorithm

2. Fit an interpretable model g , i.e., a linear regression using $\mathcal{D}_{\text{LIME}}$ and the following loss function:

$$\mathcal{L}_{\text{LIME}}(f, g, \pi_{\mathbf{x}_{\text{Bob}}}) = \sum_{i=1}^{N_{\text{Examples}}} \pi_{\mathbf{x}_{\text{Bob}}}(\mathbf{x}_{\text{Bob}}^{(i)}) \cdot \left(f(\mathbf{x}_{\text{Bob}}^{(i)}) - g(\mathbf{x}_{\text{Bob}}^{(i)}) \right)^2,$$

where $\pi_{\mathbf{x}_{\text{Bob}}}(\mathbf{x}_{\text{Bob}}^{(i)})$ represents the distance from the perturbed example $\mathbf{x}_{\text{Bob}}^{(i)}$ to the instance to be explained $\mathbf{x}_{\text{Bob}}$.

“Explaining” to Bob



The bold red cross is the instance being explained. LIME samples instances (grey nodes), gets predictions using f (gamma MDN) and weighs them by the proximity to the instance being explained (represented here by size). The dashed line g is the learned local explanation.

“Again the approximation must be imperfect, otherwise one would throw out the black box and instead use the explanation as an inherently interpretable model.” (Rudin et al., 2022)

SHAP Values

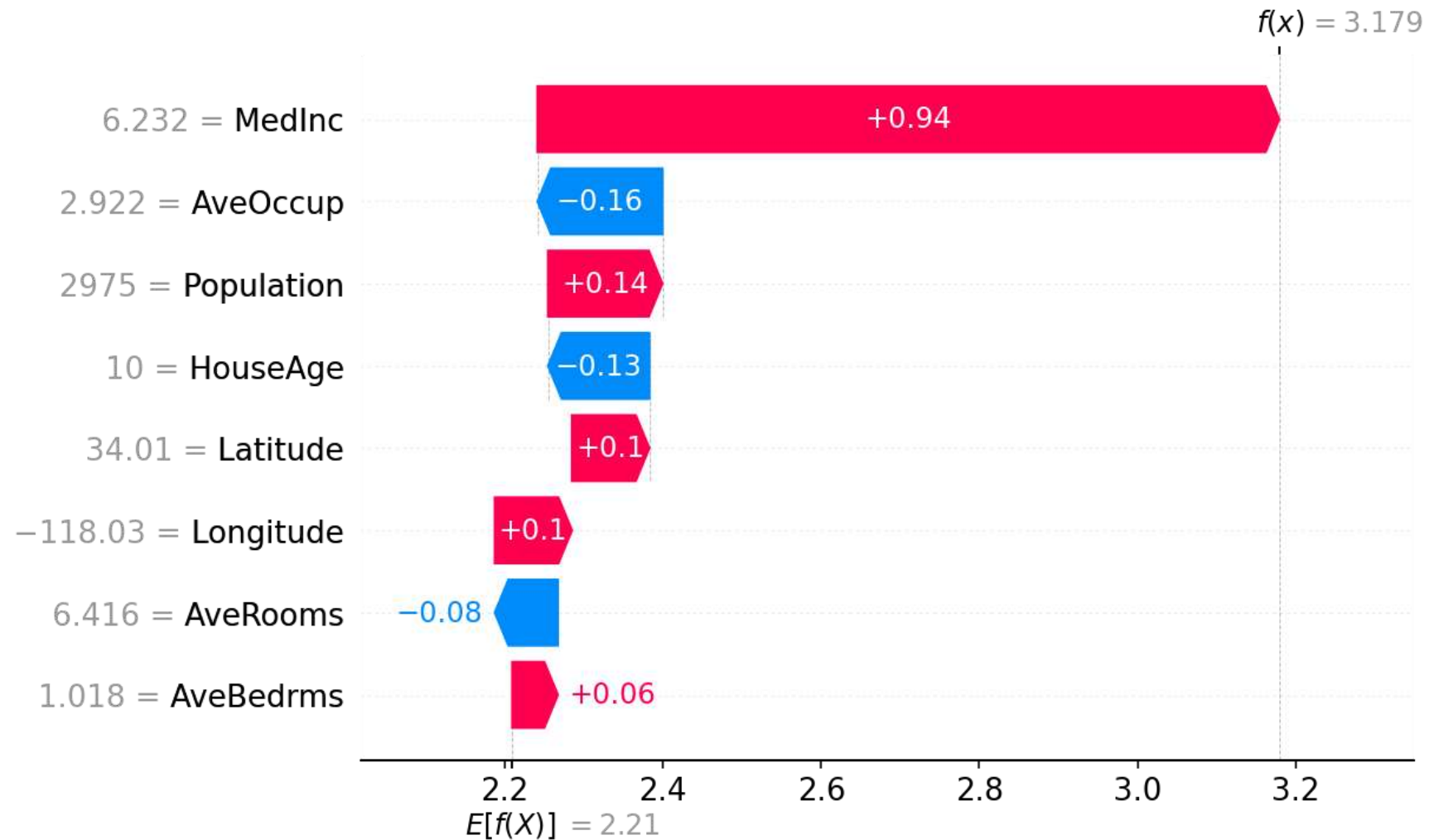
The SHapley Additive exPlanations (SHAP) value helps to quantify the contribution of each feature to the prediction for a specific instance (Lundberg & Lee, 2017).

The SHAP value for the j th feature is defined as

$$\text{SHAP}^{(j)}(\mathbf{x}) = \sum_{U \subset \{1, \dots, p\} \setminus \{j\}} \frac{1}{p} \binom{p-1}{|U|}^{-1} (\mathbb{E}[Y | \mathbf{x}^{(U \cup \{j\})}] - \mathbb{E}[Y | \mathbf{x}^{(U)}]),$$

where p is the number of features. A positive SHAP value indicates that the variable increases the prediction value.

SHAP waterfall plot



SHAP waterfall plot

Source: [shap](#) package documentation, *An introduction to explainable AI with Shapley values*

Lecture Outline

- Introduction
- Illustrative Example
- Inherently Interpretable Models
- Post-hoc Explanations
- **Belgian Motor Dataset**
- Interpreting Inherently Interpretable Models
- Neural Additive Models on the Belgian Data
- Explaining a Neural Network with Partial Dependence Plots
- Explaining Specific Models

beMTPL97 dataset

```
1 data = pd.read_csv('data/raw/beMTPL97.csv')
2 claims = data.drop(columns = ["id", "claim", "amount", "average"])
3 claims.shape
```

(163212, 14)

```
1 postcode = claims.pop("postcode")
2 train_raw, test_raw = train_test_split(claims, test_size=0.2, random_state=2000, stratify
3
4 X_train_raw = train_raw.drop(columns='nclaims')
5 X_test_raw = test_raw.drop(columns='nclaims')
6 y_train_raw = train_raw['nclaims']
7 y_test_raw = test_raw['nclaims']
8
9 int_cols = X_train_raw.select_dtypes(include='integer').columns
10 X_train_raw[int_cols] = X_train_raw[int_cols].astype(float)
11 X_test_raw[int_cols] = X_test_raw[int_cols].astype(float)
12
13 num_vars = ['expo', 'ageph', 'bm', 'power', 'agec', 'lat', 'long']
14 cat_vars = ['coverage', 'sex', 'fuel', 'use', 'fleet']
```


Covariates

Numerical variables:

Variable	Description
<code>expo</code>	exposure to risk.
<code>ageph</code>	policyholder's age.
<code>bm</code>	An integer for the level on the former Belgian bonus-malus scale (0 to 22; higher = worse claims history).
<code>power</code>	vehicle's horsepower in kilowatts.
<code>agec</code>	vehicle's age in years.
<code>long</code>	longitude of the policyholder's municipality center.
<code>lat</code>	latitude of the policyholder's municipality center.

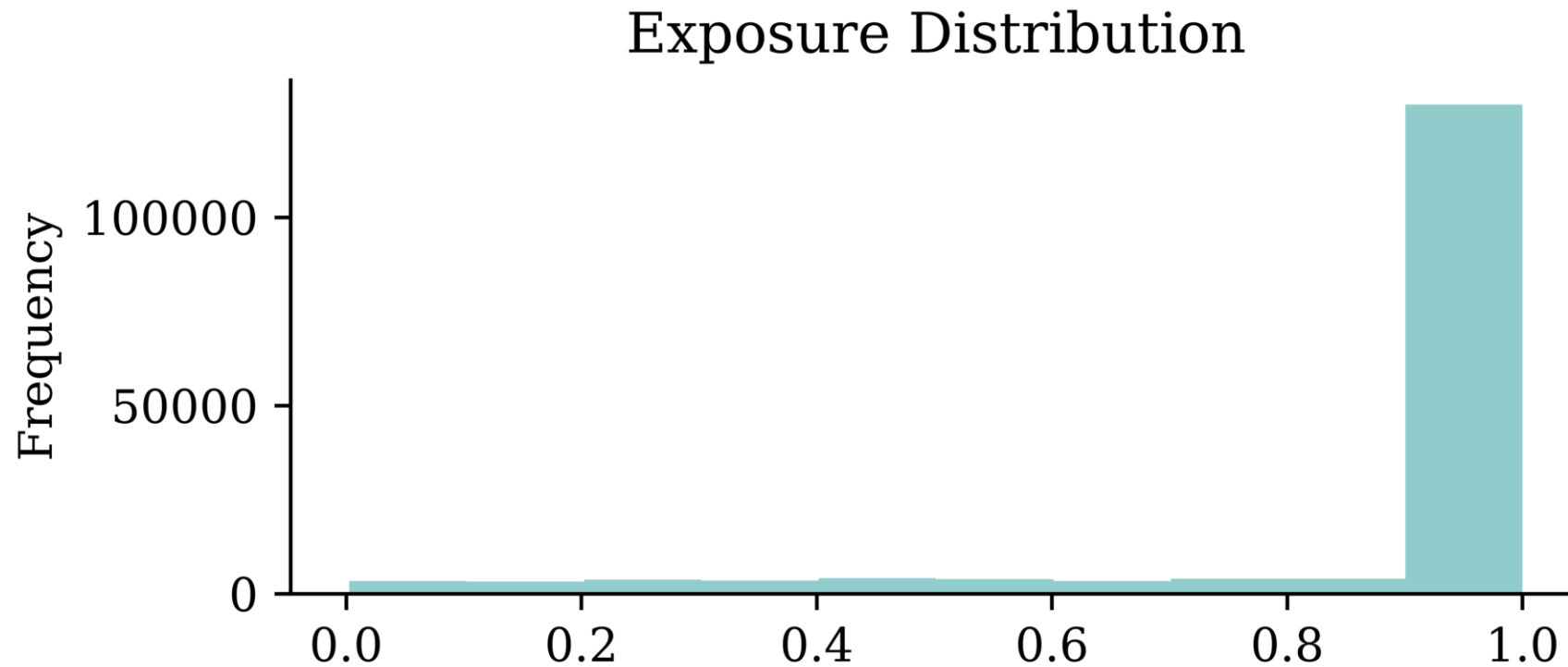
Target is `nclaims`.

Categorical variables:

Variable	Description
<code>coverage</code>	insurance coverage level: " <code>TPL</code> " (third party liability), " <code>TPL+</code> " (TPL + limited material damage), " <code>TPL++</code> " (TPL + comprehensive material damage).
<code>sex</code>	policyholder's gender: " <code>female</code> ", " <code>male</code> ".
<code>bm</code>	level on the former Belgian bonus-malus scale (0 to 22; higher = worse claims history).
<code>fuel</code>	vehicle's fuel type: " <code>gasoline</code> " or " <code>diesel</code> ".
<code>use</code>	vehicle's use: " <code>private</code> " or " <code>work</code> ".
<code>fleet</code>	vehicle is part of a fleet (<code>1</code> = yes, <code>0</code> = no).

Exposure & frequency

```
1 claims["expo"].plot(kind='hist', title='Exposure Distribution')
```

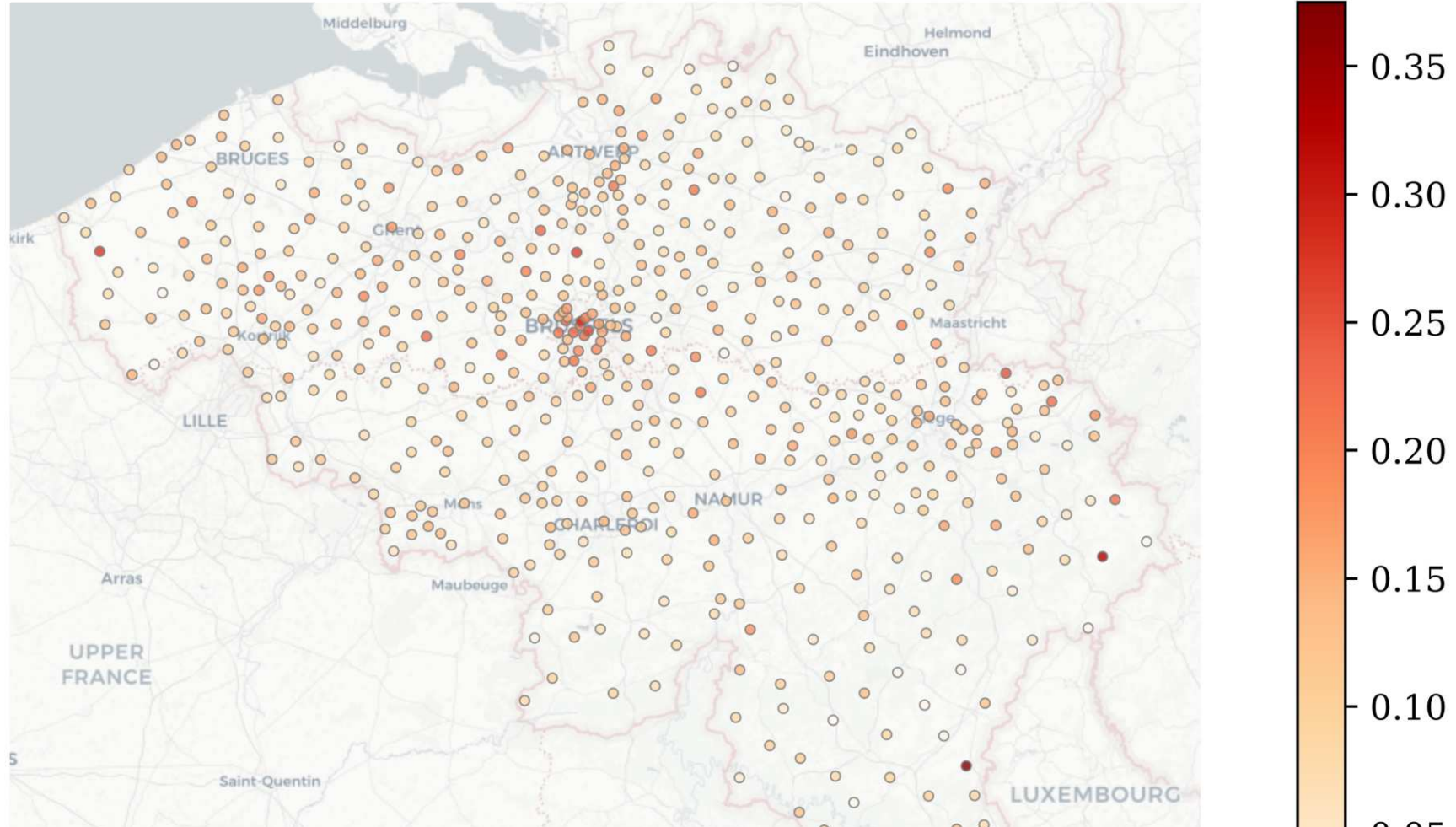


```
1 claims["nclaims"].value_counts()
```

```
nclaims
0      144936
1       16539
2        1556
```

Map of claims

Average Claims per Location



Fitting using statsmodels

```

1 formula = "nclaims ~ expo + ageph + bm + power + agec + lat + long " + \
2         " + C(coverage) + C(sex) + C(fuel) + C(use) + C(fleet)"
3
4 glm_model = smf.glm(
5     formula=formula,
6     data=train_raw,
7     family=sm.families.Poisson()
8 ).fit()

```

i Question

What do you expect to be the relationship between `ageph` and `nclaims`?

```

1 X_train_first = X_train_raw.iloc[0:1].copy()
2 X_train_first

```

	expo	coverage	ageph	sex	bm	power	agec	fuel	use	fleet
25776	1.0	TPL	59.0	female	0.0	51.0	15.0	gasoline	private	0.0

Summary

```
1 glm_model.summary().tables[0]
```

Generalized Linear Model Regression Results

Dep. Variable:	nclaims	No. Observations:	130569
Model:	GLM	Df Residuals:	130555
Model Family:	Poisson	Df Model:	13
Link Function:	Log	Scale:	1.0000
Method:	IRLS	Log-Likelihood:	-49908.
Date:	Mon, 06 Jul 2026	Deviance:	69690.
Time:	16:50:47	Pearson chi2:	1.39e+05
No. Iterations:	6	Pseudo R-squ. (CS):	0.01842
Covariance Type:	nonrobust		

Summary

```
1 glm_model.summary().tables[1]
```

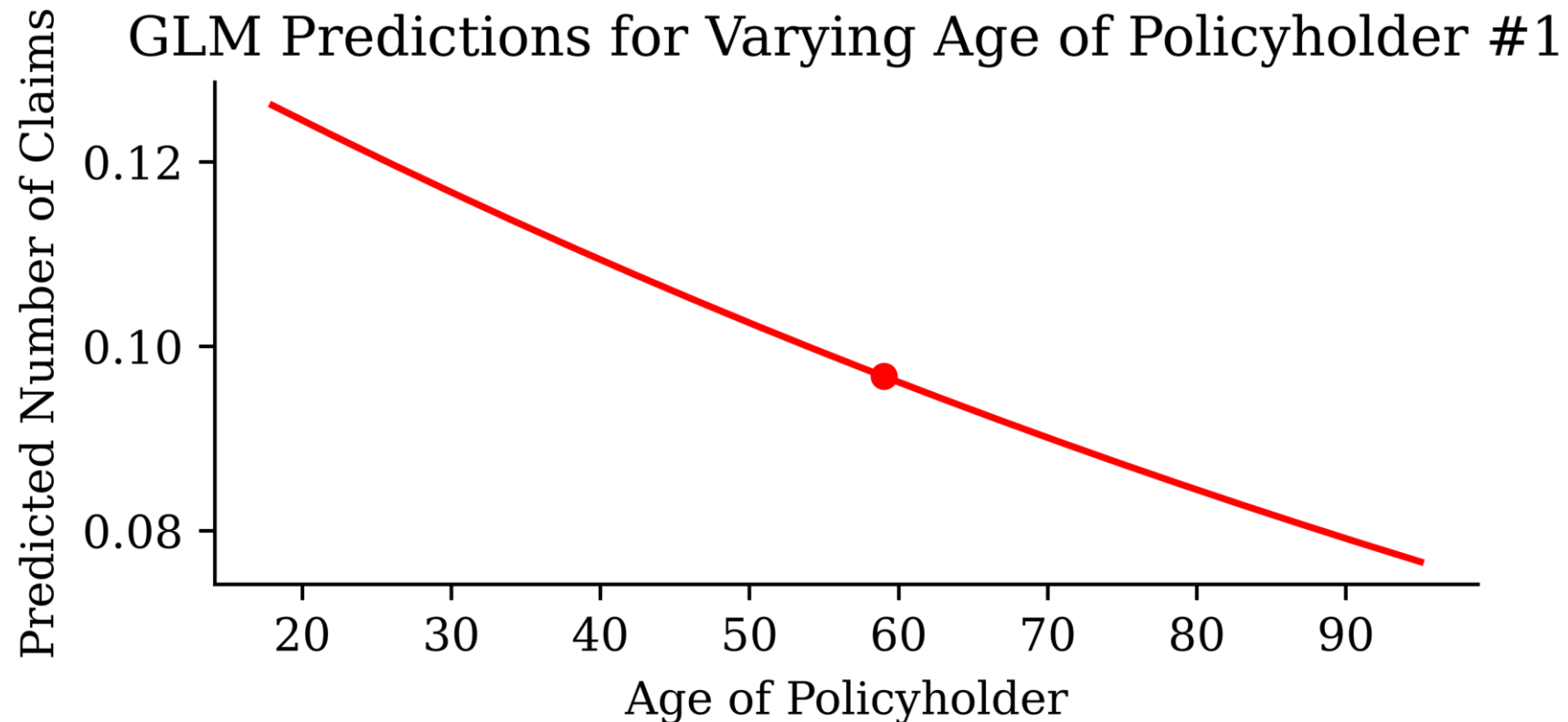
	coef	std err	z	P> z	[0.025	0.975]
Intercept	-7.0204	1.344	-5.222	0.000	-9.655	-4.386
C(coverage)[T.TPL+]	-0.0765	0.020	-3.889	0.000	-0.115	-0.038
C(coverage)[T.TPL++]	-0.0715	0.027	-2.660	0.008	-0.124	-0.019
C(sex)[T.male]	-0.0259	0.018	-1.429	0.153	-0.061	0.010
C(fuel)[T.gasoline]	-0.1793	0.017	-10.459	0.000	-0.213	-0.146
C(use)[T.work]	-0.0864	0.037	-2.306	0.021	-0.160	-0.013
C(fleet)[T.1]	-0.0886	0.048	-1.832	0.067	-0.183	0.006
expo	0.9893	0.041	24.196	0.000	0.909	1.069
ageph	-0.0065	0.001	-10.724	0.000	-0.008	-0.005
bm	0.0642	0.002	33.035	0.000	0.060	0.068
power	0.0036	0.000	8.309	0.000	0.003	0.004
agec	-0.0019	0.002	-0.872	0.383	-0.006	0.002
lat	0.0777	0.026	2.969	0.003	0.026	0.129
long	0.0279	0.011	2.542	0.011	0.006	0.049

Lecture Outline

- Introduction
- Illustrative Example
- Inherently Interpretable Models
- Post-hoc Explanations
- Belgian Motor Dataset
- **Interpreting Inherently Interpretable Models**
- Neural Additive Models on the Belgian Data
- Explaining a Neural Network with Partial Dependence Plots
- Explaining Specific Models

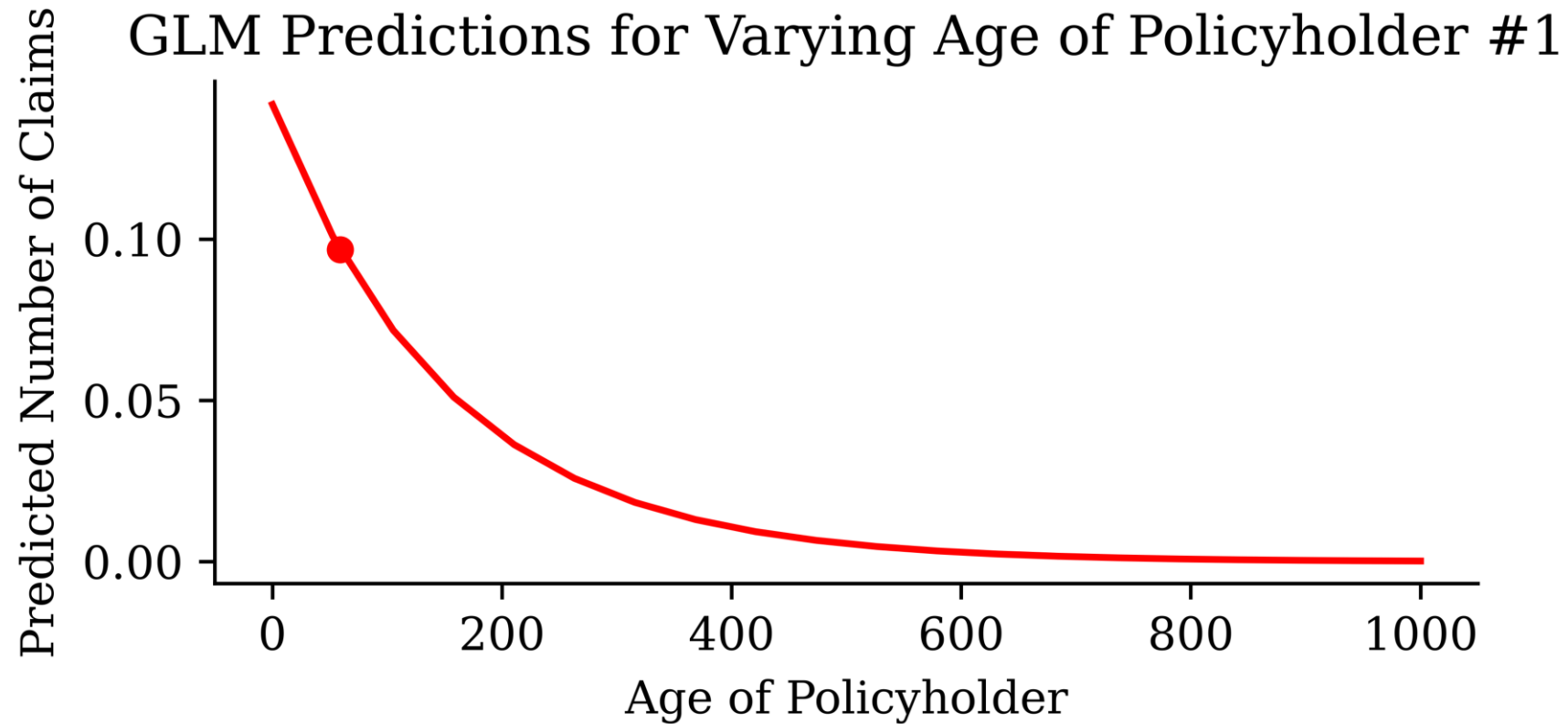
GLM Ceteris Paribus Plots

```
1 ageph_range = np.linspace(train_raw['ageph'].min(), train_raw['ageph'].max(), 20)
2 y_pred_batch = []
3 for ageph in ageph_range:
4     X_train_first['ageph'] = ageph
5     y_pred_batch.append(glm_model.predict(X_train_first))
```

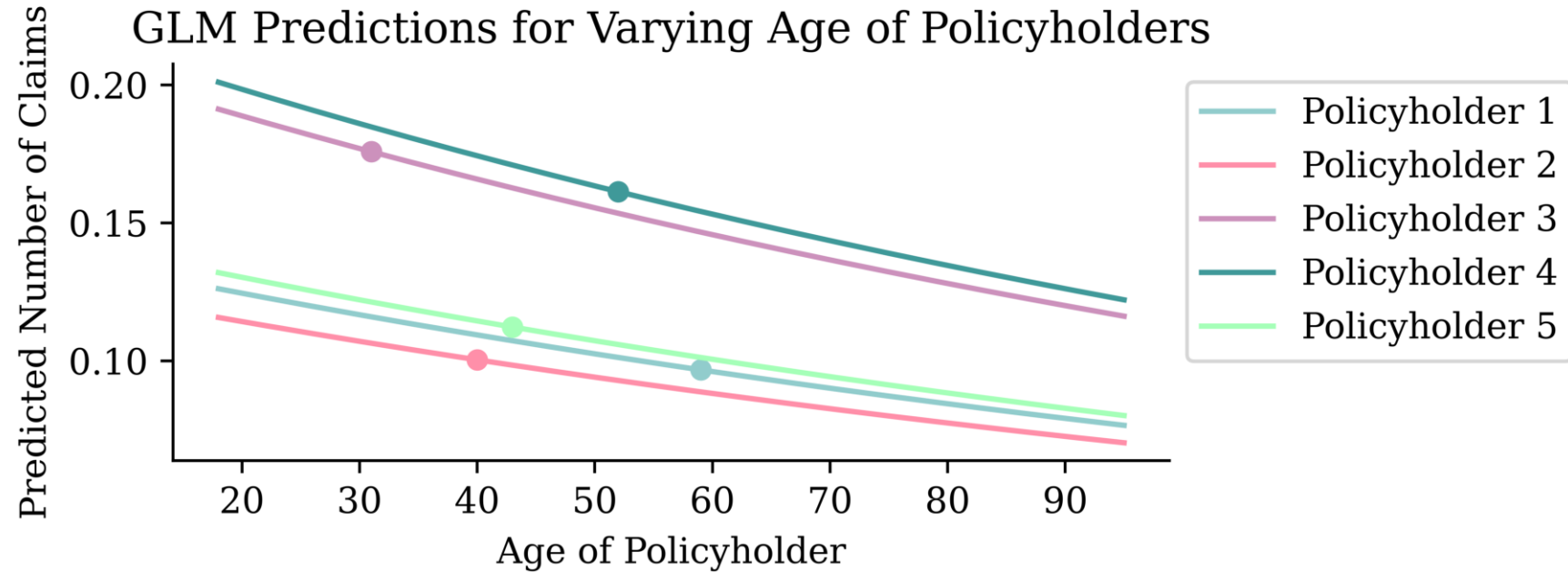


Zoomed out

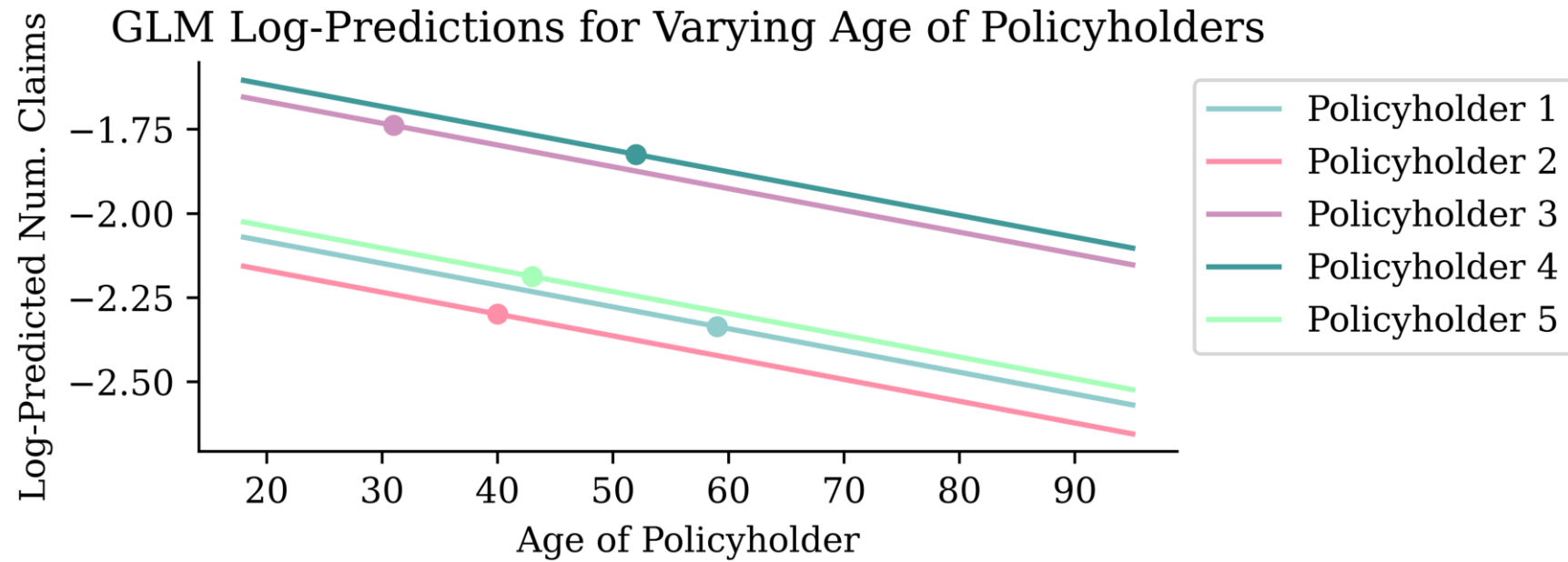
We can see the trend if we zoom out to ages between 0 & 1,000.



What if we look at multiple policyholders?

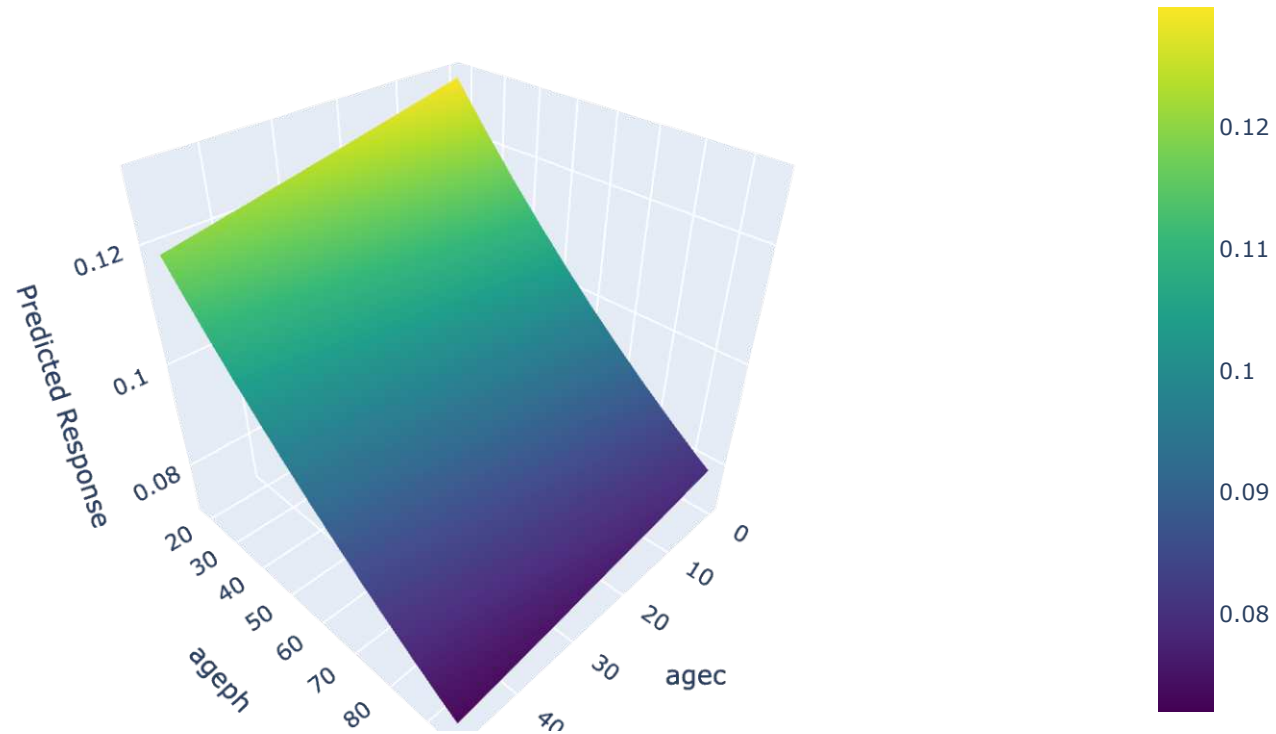


Logarithmic scale



Do it for `ageph` and `agec`

GLM-predicted Number of Claims



Generalised Additive Models (GAMs)

Transform a GLM's inputs nonlinearly, i.e.,

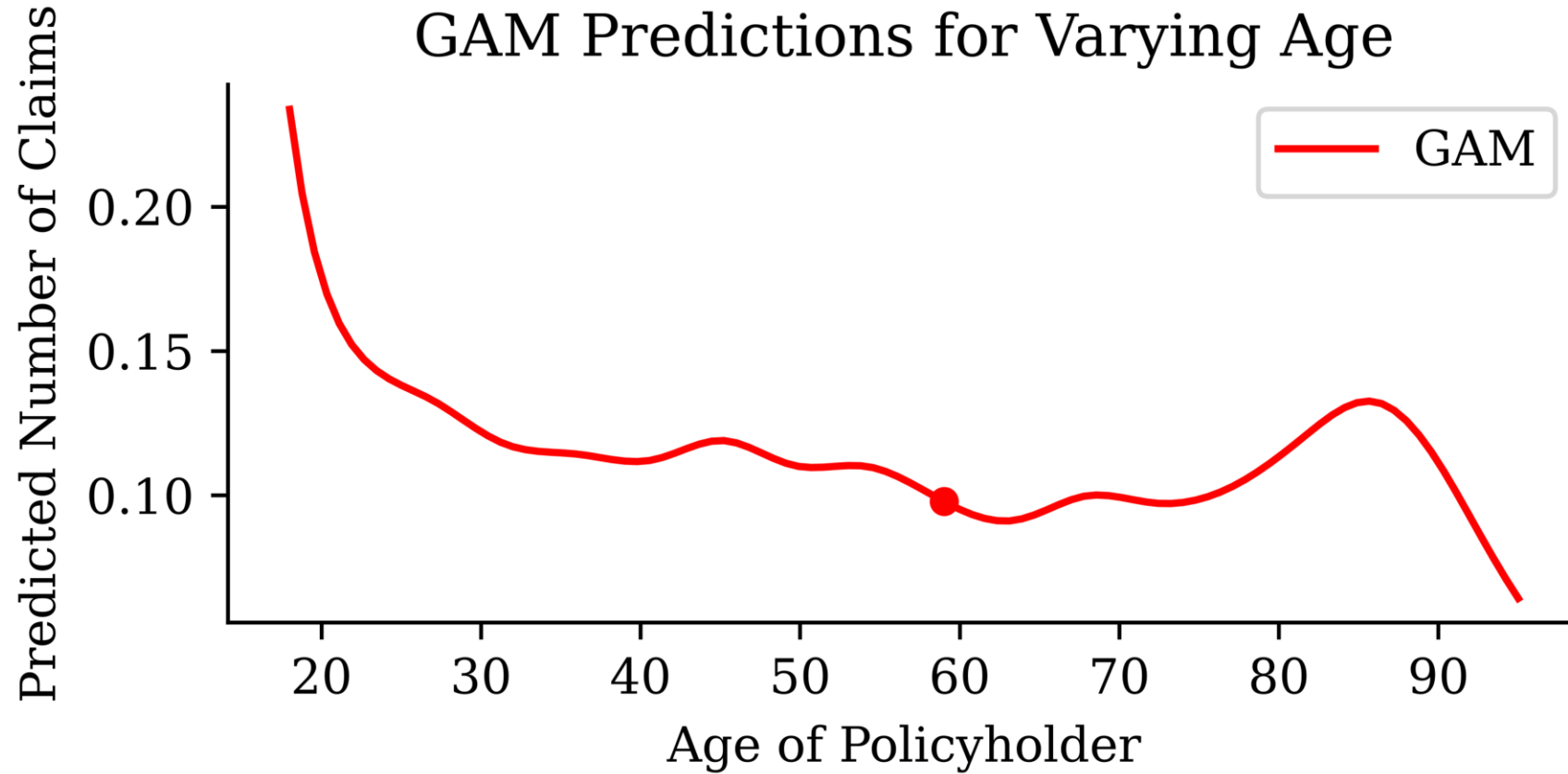
$$\hat{y}_i = g^{-1}(\beta_0 + f_1(x_{i,1}) + f_2(x_{i,2}) + \dots + f_p(x_{i,p}))$$

The f_j are typically polynomials, step functions, or splines.

```
1 ct = make_column_transformer(
2     ("passthrough", num_vars),
3     (OrdinalEncoder(), cat_vars),
4     verbose_feature_names_out = False
5 )
6 X_train_gam = ct.fit_transform(X_train_raw)
7 X_test_gam = ct.transform(X_test_raw)
8 y_train_gam = y_train_raw.values
9 y_test_gam = y_test_raw.values
```

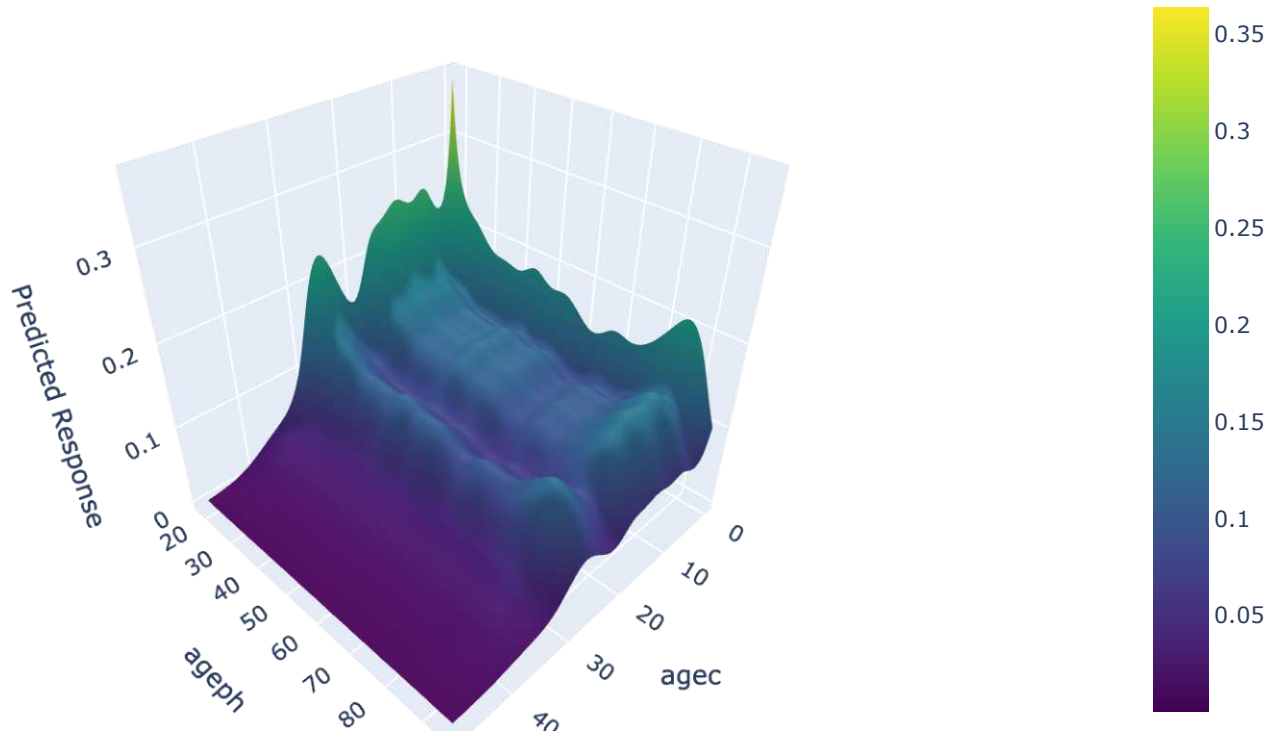
```
1 formula = s(0) + s(1) + s(2) + s(3) + s(4) + s(5) + s(6) + \
2     f(7) + f(8) + f(9) + f(10) + f(11)
3 gam_model = PoissonGAM(formula).fit(X_train_gam, y_train_gam)
```

Ceteris paribus plot for `ageph`



GAM bivariate effects for `ageph` and `agec`

GLM-predicted Number of Claims



Lecture Outline

- Introduction
- Illustrative Example
- Inherently Interpretable Models
- Post-hoc Explanations
- Belgian Motor Dataset
- Interpreting Inherently Interpretable Models
- **Neural Additive Models on the Belgian Data**
- Explaining a Neural Network with Partial Dependence Plots
- Explaining Specific Models

From GAM to Neural Additive Model

A *Neural Additive Model* (NAM) is a GAM where each f_j is a small neural network.

$$\hat{y}_i = g^{-1}(\beta_0 + f_1(x_{i,1}) + f_2(x_{i,2}) + \cdots + f_p(x_{i,p}))$$

The `anam` package (*Actuarial NAM*) fits these with a Poisson/Gamma likelihood, an exposure offset, optional *monotonicity* constraints, and automatic selection of features and pairwise interactions.

Specify the model

`anam` works on the raw data: continuous vs categorical is read off the dtypes, and the standardising/encoding happens inside the model. A `smoothness` penalty keeps the continuous shape functions from overfitting.

```

1 from anam import ANAM
2
3 # Carve a validation set out of the training data
4 anam_train, anam_val = train_test_split(train_raw, test_size=0.25, random_state=2000)
5
6 model_anam = ANAM(
7     distribution="poisson",          # implies log link + Poisson NLL
8     exposure="expo",                # offset: log(expo) added to the linear predictor
9     monotone={"bm": "increasing"},  # bonus-malus effect must be increasing
10    smoothness=1e-3,                 # roughness penalty on continuous shape functions
11 )

```

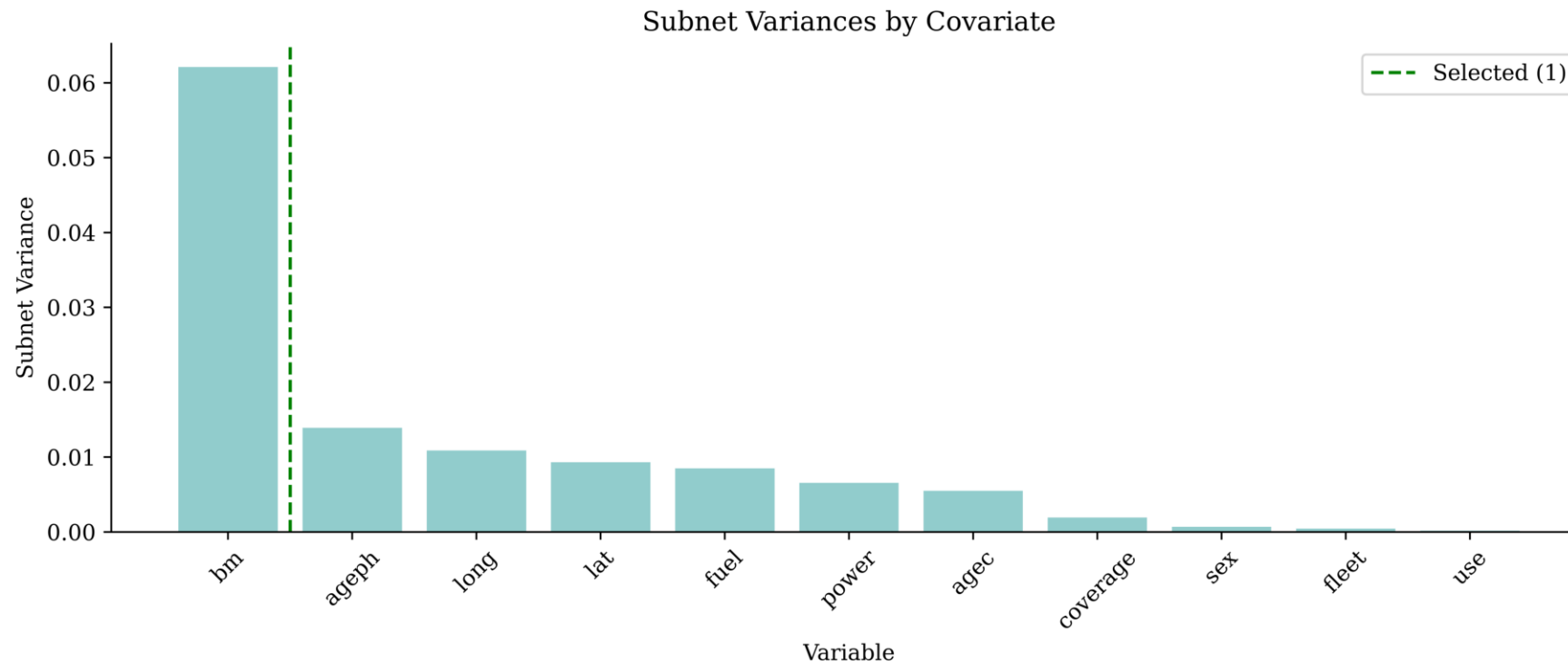
The paper's workflow is *assess, then select*: rank every candidate feature, read the scree plot, and only then decide how many to keep. (For a hands-off fit, skip straight to `model_anam.fit(...)`, which runs the same screening and picks the cutoffs automatically with an elbow rule.)

Pick the main effects

`rank_main_effects` trains an ensemble of shallow NAMs and scores each covariate by the *variance* of its shape function — a proxy for how much signal it carries — without yet committing to a cutoff.

```
1 model_anam.rank_main_effects(anam_train, anam_train["nclaims"],  
2   X_val=anam_val, y_val=anam_val["nclaims"], random_state=2000)
```

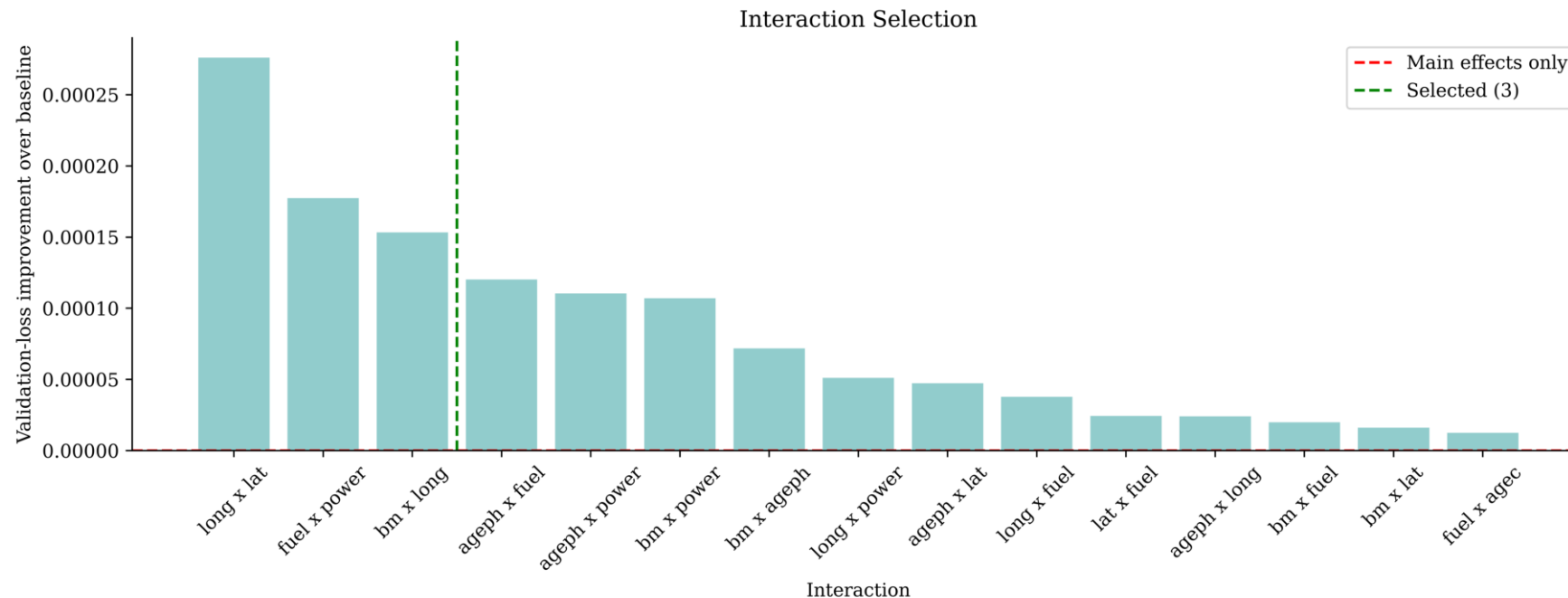
```
1 model_anam.plot_main_effect_importance()
```



Pick the interactions

`rank_interactions` scores every pairwise term that passes the *strong heredity* rule (a pair is only considered if both of its main effects are in the top n_{main}), measuring how much each candidate improves the validation loss over the mains-only baseline.

```
1 model_anam.rank_interactions(anam_train, anam_train["nclaims"],
2   X_val=anam_val, y_val=anam_val["nclaims"],
3   n_main=7, random_state=2000)
```



Lock in choices and fit

```
1 model_anam.choose(n_main=7, n_interactions=6)
```

```
1 model_anam.features_
```

```
['bm', 'ageph', 'long', 'lat', 'fuel', 'power', 'agec']
```

```
1 model_anam.interactions_
```

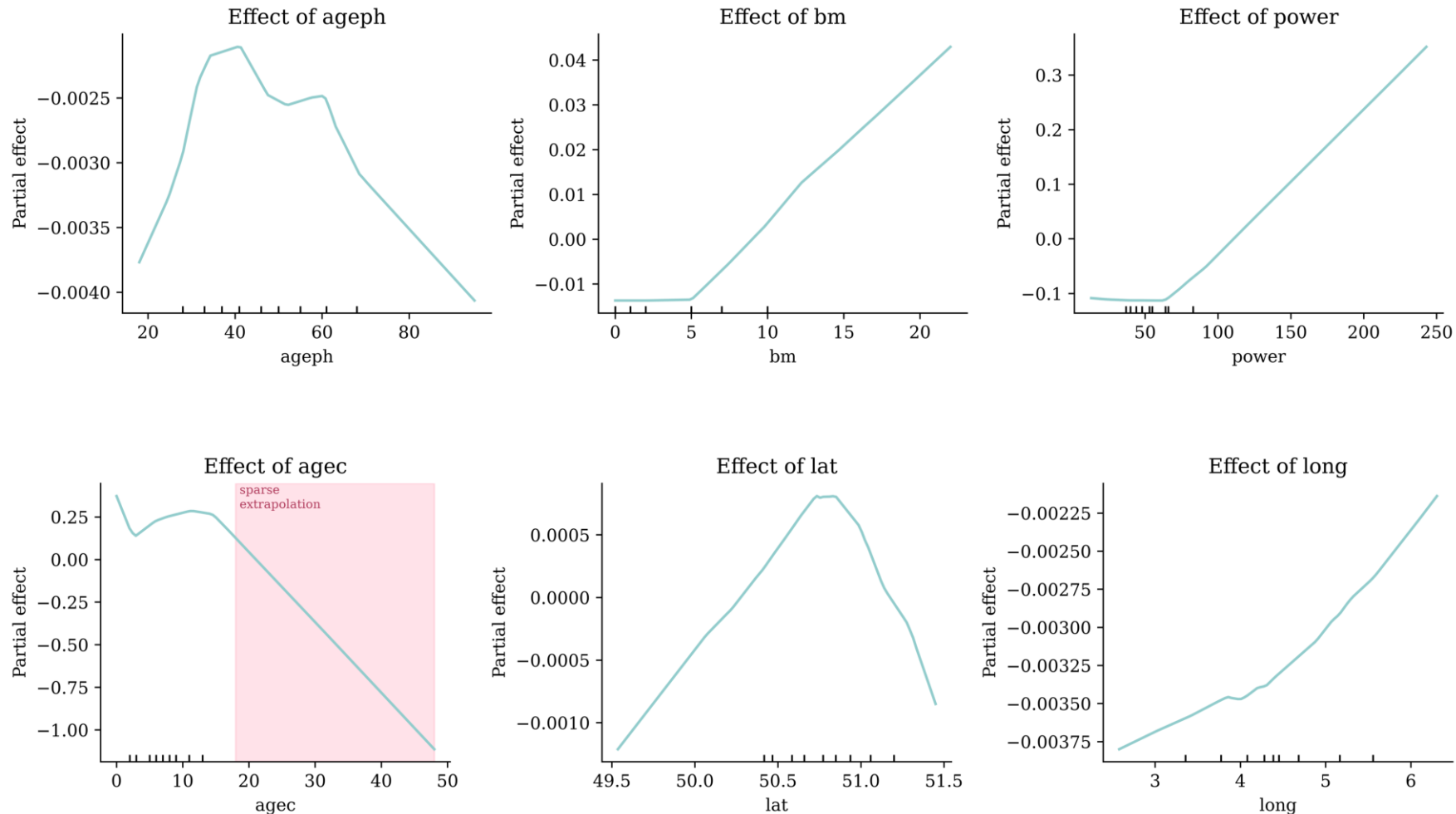
```
[('long', 'lat'),  
 ('fuel', 'power'),  
 ('bm', 'long'),  
 ('ageph', 'fuel'),  
 ('ageph', 'power'),  
 ('bm', 'power')]
```

Exposure enters the final fit as an *offset*, not a feature: $\log \mu = \beta_0 + \sum_j f_j(x_j) + \log(\text{expo})$.

```
1 model_anam.fit(anam_train, anam_train["nclaims"],  
2               X_val=anam_val, y_val=anam_val["nclaims"],  
3               epochs=300, batch_size=5000, lr=0.01, patience=40, random_state=2000)
```

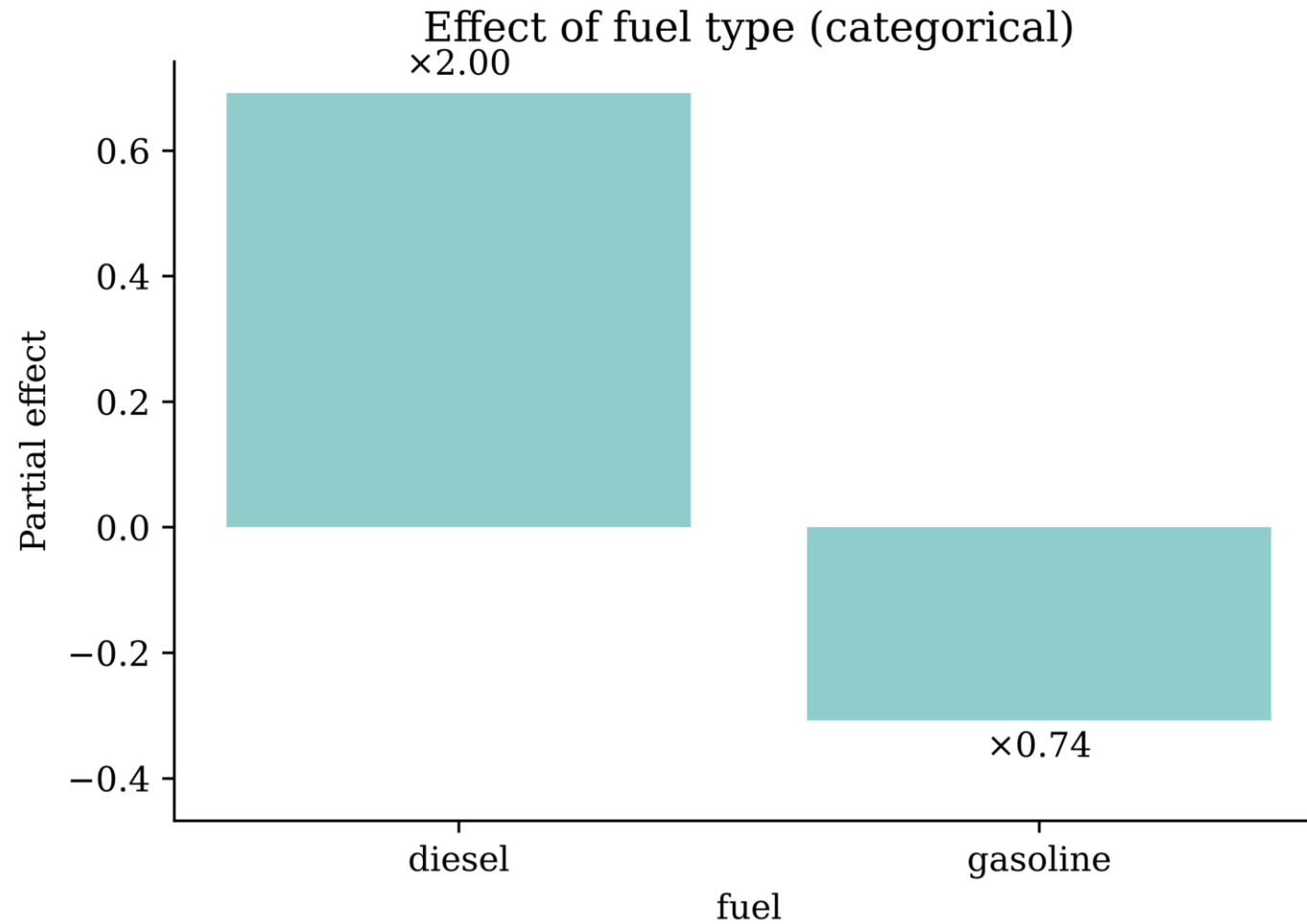
Reading the shape functions

Each curve is that feature's contribution to $\log \mu$; back on the original scale, $\exp(f_j)$ is a *multiplier* on the expected claim count.



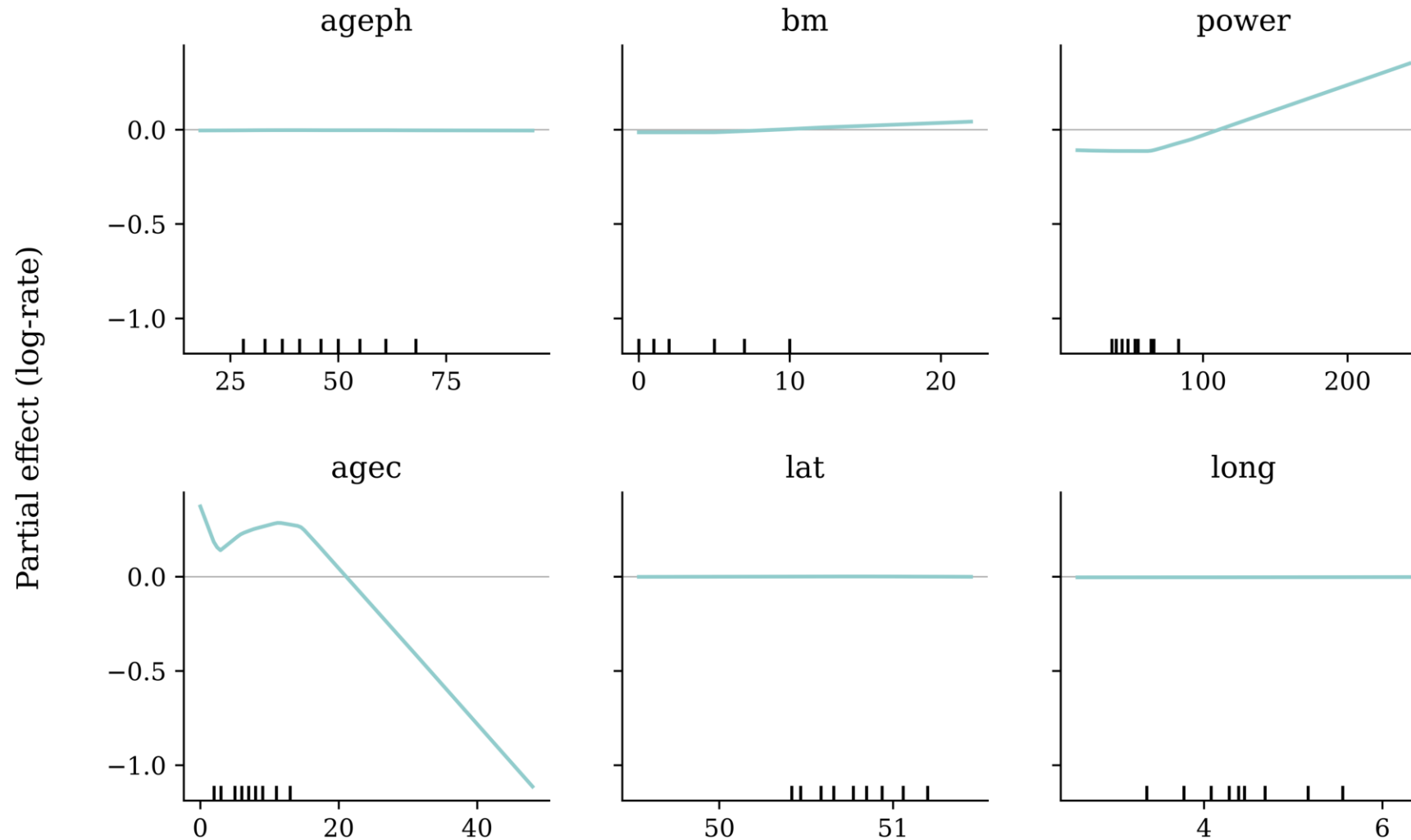
A categorical effect: fuel

Continuous features give *curves*; a categorical feature gives one value per level. `anam` draws it as a bar chart with the real category labels, still in log-rate units.



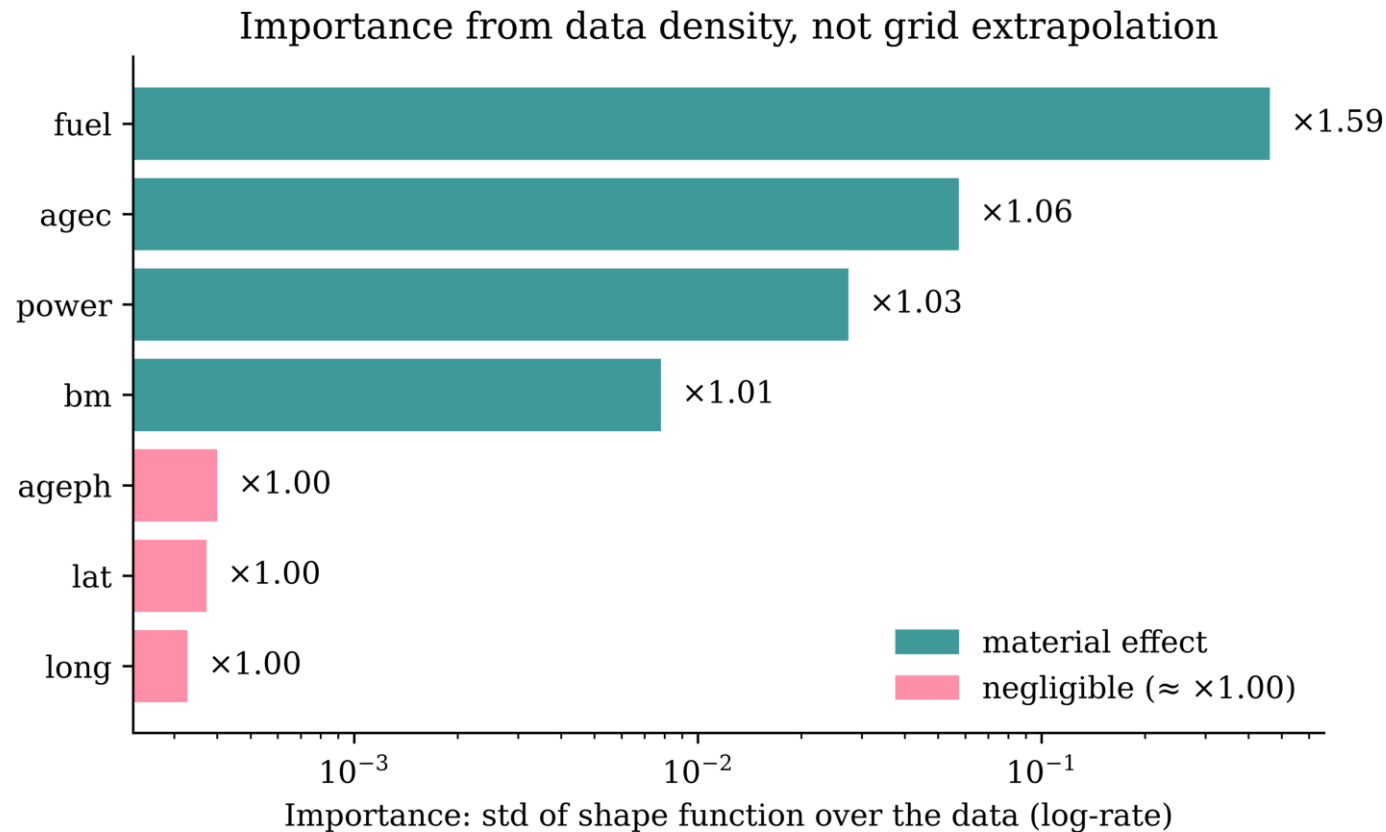
The same curves, one honest scale

Each panel above had its *own* y-axis. Put the six continuous effects on a *shared* scale and the picture changes: **agec** and **power** swing hard, while **ageph**, **lat** and **long** flatten to near-zero lines.

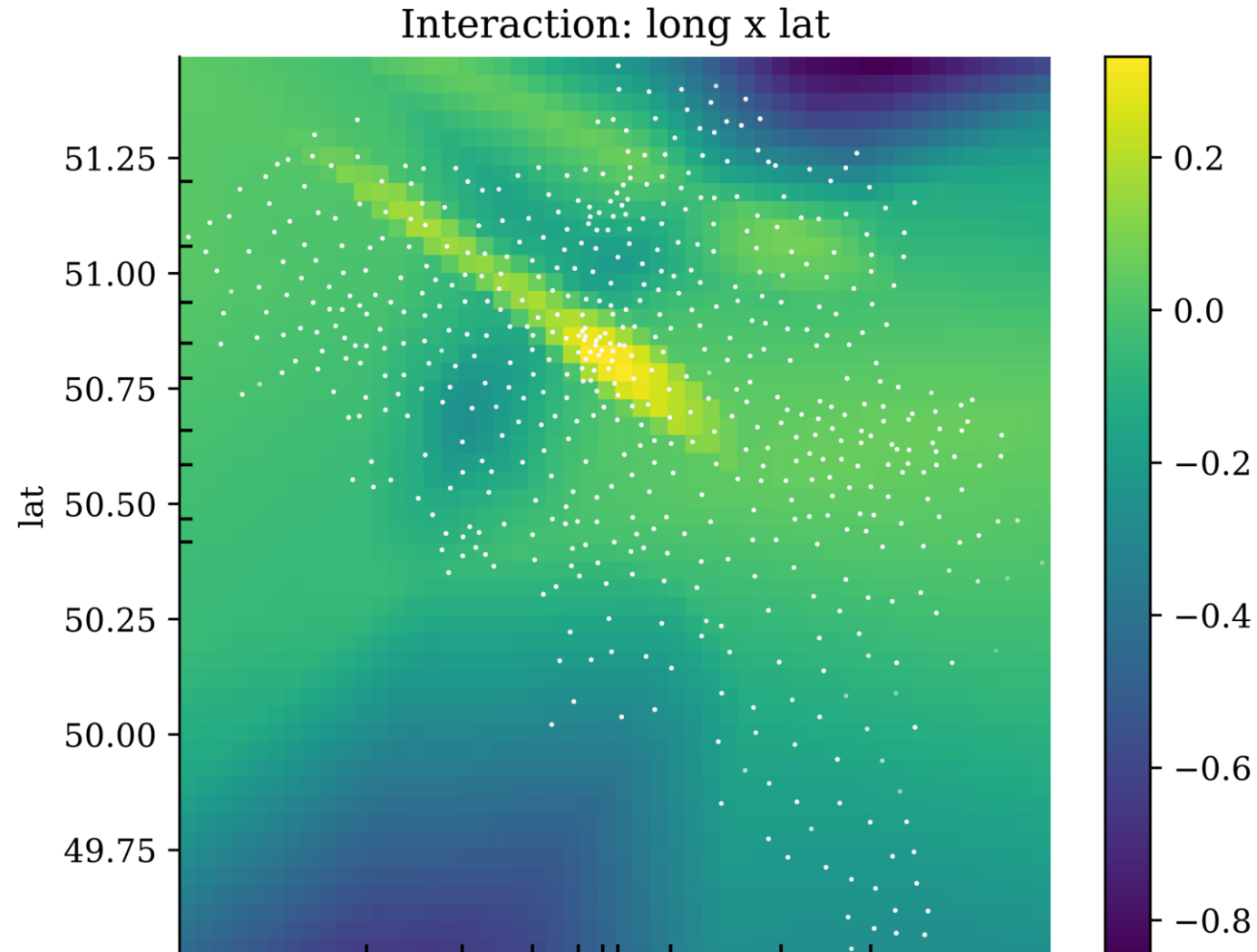


Which effects actually matter

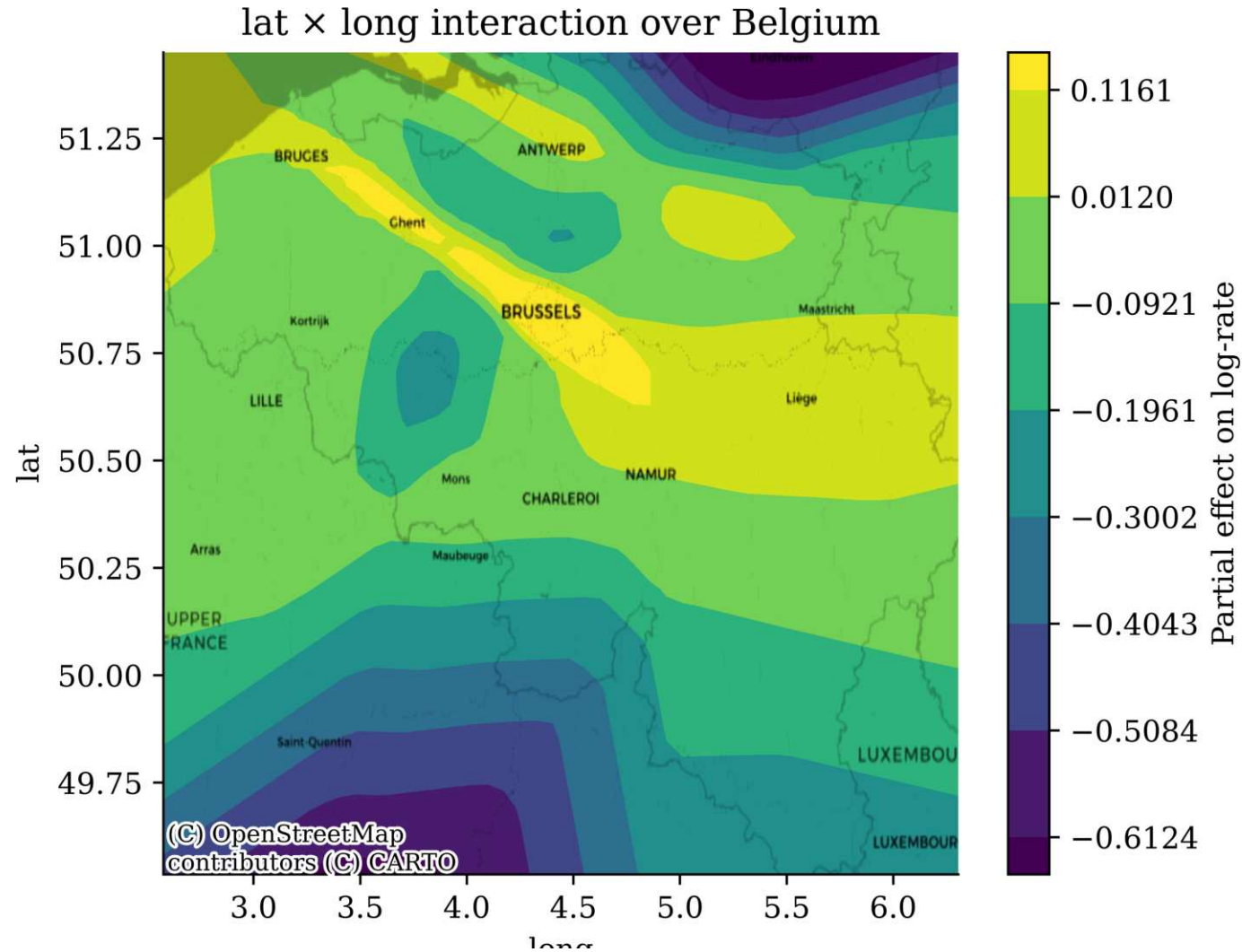
Reading a swing straight off the *grid* is unfair: it rewards steep extrapolation into empty regions (like old [agec](#)). Instead measure the *standard deviation of each shape function across the training data* — the variance-based importance [anam](#) itself uses. Weighting by where the data really is, exponentiating gives a *typical multiplier* on the expected claim frequency.



An interaction effect



The interaction over a map



Lecture Outline

- Introduction
- Illustrative Example
- Inherently Interpretable Models
- Post-hoc Explanations
- Belgian Motor Dataset
- Interpreting Inherently Interpretable Models
- Neural Additive Models on the Belgian Data
- **Explaining a Neural Network with Partial Dependence Plots**
- Explaining Specific Models

Build a neural network

```

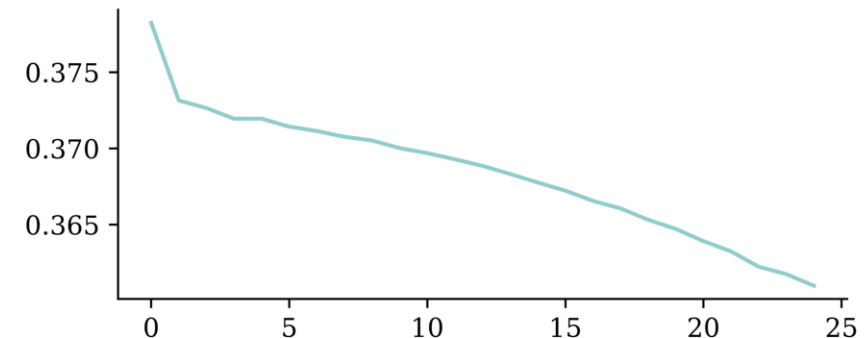
1 ct = make_column_transformer(
2     (StandardScaler(), num_vars),
3     (OneHotEncoder(drop="first", sparse_c
4     verbose_feature_names_out=False
5 )
6 ct.fit(X_train_raw)
7
8 X_train_nn = ct.transform(X_train_raw)
9 X_test_nn = ct.transform(X_test_raw)
10 y_train_nn = y_train_raw.values
11 y_test_nn = y_test_raw.values
12
13
14 random.seed(123)
15
16 nn_model = Sequential(
17     [
18         Input(shape=(X_train_nn.shape[1],
19         Dense(128, activation="relu"),
20         Dense(128, activation="relu"),
21         Dense(128, activation="relu"),
22         Dense(1, activation="exponential"
23     ]
24 )

```

```

1 nn_model.compile(
2     optimizer="adam",
3     loss="poisson",
4     metrics=["mae"]
5 )
6
7 history = nn_model.fit(X_train_nn, y_train_nn,
8     epochs=25, batch_size=64, verbose=0,
9 )

```



Metrics

```

1  # Evaluate the neural network model
2  y_pred_nn = nn_model.predict(X_test_nn, verbose=0, batch_size=X_test_nn.shape[0]).flatten
3  print(f"NN mean Poisson deviance : {mean_poisson_deviance(y_test_nn, y_pred_nn):.4f}")
4  print(f"NN MAE                    : {mean_absolute_error(y_test_nn, y_pred_nn):.4f}")
5
6  # Compare to the GAM
7  y_pred_gam = gam_model.predict(X_test_gam).ravel()
8  print(f"GAM mean Poisson deviance : {mean_poisson_deviance(y_test_gam, y_pred_gam):.4f}")
9  print(f"GAM MAE                    : {mean_absolute_error(y_test_gam, y_pred_gam):.4f}")
10
11 ## Compare to the GLM
12 y_pred_glm = glm_model.predict(X_test_raw).values.ravel()
13 print(f"GLM mean Poisson deviance : {mean_poisson_deviance(y_test_raw, y_pred_glm):.4f}")
14 print(f"GLM MAE                    : {mean_absolute_error(y_test_raw, y_pred_glm):.4f}")

```

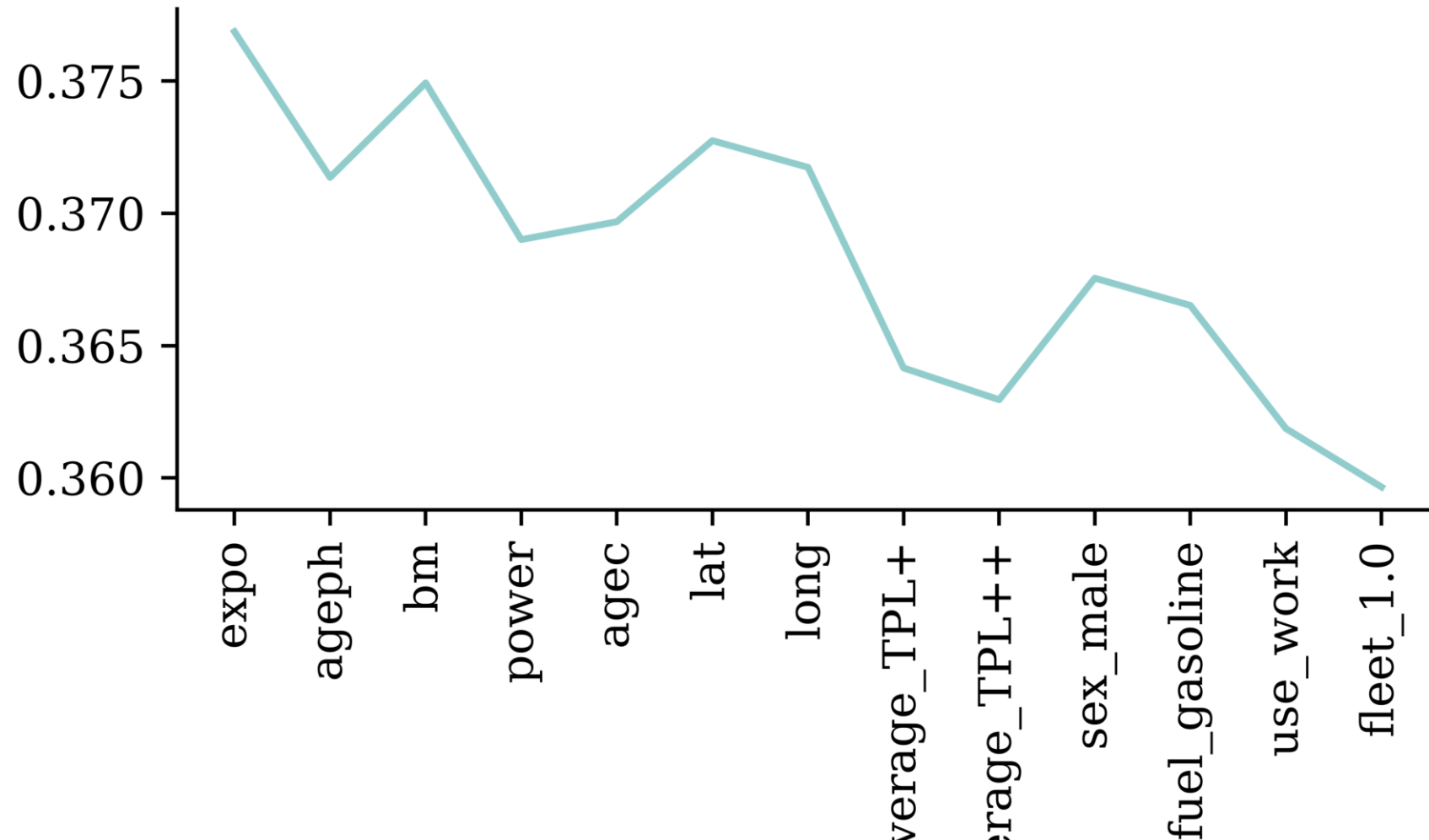
```

NN mean Poisson deviance : 0.5468
NN MAE                    : 0.2174
GAM mean Poisson deviance : 0.5254
GAM MAE                    : 0.2149
GLM mean Poisson deviance : 0.5326
GLM MAE                    : 0.2164

```

Permutation importance example

```
1 scores = permutation_test(nn_model, X_train_nn.values, y_train_nn)
2 plt.plot(scores[:,0], label='Loss')
3 plt.xticks(ticks=np.arange(len(X_train_nn.columns)), labels=X_train_nn.columns, rotation=
```



PDP: Training data

Partial dependence plots start by looking at the training data.

```
1 X_train_raw
```

	expo	coverage	ageph	sex	bm	power	agec	fuel	use	fleet	long	lat
25776	1.000000	TPL	59.0	female	0.0	51.0	15.0	gasoline	private	0.0	4.387146	51.216042
63185	0.819178	TPL++	40.0	female	0.0	96.0	3.0	gasoline	private	0.0	5.500567	50.583188
130175	1.000000	TPL	31.0	female	8.0	40.0	8.0	gasoline	private	0.0	3.721116	50.535314
...	...	...	...	...	...	...	...	...	...	...	...	...
24617	1.000000	TPL	75.0	male	0.0	29.0	17.0	gasoline	private	0.0	4.387146	51.216042
61581	1.000000	TPL	63.0	male	0.0	55.0	11.0	gasoline	private	0.0	5.612566	50.680020
10699	1.000000	TPL+	50.0	male	6.0	74.0	3.0	gasoline	private	0.0	4.678745	50.687562

130569 rows × 12 columns

```
1 nn_model.predict(ct.transform(X_train_raw), verbose=0).mean()
```

```
np.float32(0.12718096)
```


PDP: Alternate Reality

```
1 X_train_pd = X_train_raw.copy()
2 X_train_pd['ageph'] = 18
3 X_train_pd
```

	expo	coverage	ageph	sex	bm	power	agec	fuel	use	fleet	long	lat
25776	1.000000	TPL	18	female	0.0	51.0	15.0	gasoline	private	0.0	4.387146	51.216042
63185	0.819178	TPL++	18	female	0.0	96.0	3.0	gasoline	private	0.0	5.500567	50.583188
130175	1.000000	TPL	18	female	8.0	40.0	8.0	gasoline	private	0.0	3.721116	50.535314
...	...	...	...	...	...	...	...	...	...	...	...	...
24617	1.000000	TPL	18	male	0.0	29.0	17.0	gasoline	private	0.0	4.387146	51.216042
61581	1.000000	TPL	18	male	0.0	55.0	11.0	gasoline	private	0.0	5.612566	50.680020
10699	1.000000	TPL+	18	male	6.0	74.0	3.0	gasoline	private	0.0	4.678745	50.687562

130569 rows × 12 columns

```
1 nn_model.predict(ct.transform(X_train_pd), verbose=0).mean()
```

np.float32(0.17916383)

PDP: Alternate Reality

```
1 X_train_pd = X_train_raw.copy()
2 X_train_pd['ageph'] = 19
3 X_train_pd
```

	expo	coverage	ageph	sex	bm	power	agec	fuel	use	fleet	long	lat
25776	1.000000	TPL	19	female	0.0	51.0	15.0	gasoline	private	0.0	4.387146	51.216042
63185	0.819178	TPL++	19	female	0.0	96.0	3.0	gasoline	private	0.0	5.500567	50.583188
130175	1.000000	TPL	19	female	8.0	40.0	8.0	gasoline	private	0.0	3.721116	50.535314
...	...	...	...	...	...	...	...	...	...	...	...	...
24617	1.000000	TPL	19	male	0.0	29.0	17.0	gasoline	private	0.0	4.387146	51.216042
61581	1.000000	TPL	19	male	0.0	55.0	11.0	gasoline	private	0.0	5.612566	50.680020
10699	1.000000	TPL+	19	male	6.0	74.0	3.0	gasoline	private	0.0	4.678745	50.687562

130569 rows × 12 columns

```
1 nn_model.predict(ct.transform(X_train_pd), verbose=0).mean()
```

np.float32(0.17551786)

PDP: Alternate Reality

```
1 X_train_pd = X_train_raw.copy()
2 X_train_pd['ageph'] = 90
3 X_train_pd
```

	expo	coverage	ageph	sex	bm	power	agec	fuel	use	fleet	long	lat
25776	1.000000	TPL	90	female	0.0	51.0	15.0	gasoline	private	0.0	4.387146	51.216042
63185	0.819178	TPL++	90	female	0.0	96.0	3.0	gasoline	private	0.0	5.500567	50.583188
130175	1.000000	TPL	90	female	8.0	40.0	8.0	gasoline	private	0.0	3.721116	50.535314
...	...	...	...	...	...	...	...	...	...	...	...	...
24617	1.000000	TPL	90	male	0.0	29.0	17.0	gasoline	private	0.0	4.387146	51.216042
61581	1.000000	TPL	90	male	0.0	55.0	11.0	gasoline	private	0.0	5.612566	50.680020
10699	1.000000	TPL+	90	male	6.0	74.0	3.0	gasoline	private	0.0	4.678745	50.687562

130569 rows × 12 columns

```
1 nn_model.predict(ct.transform(X_train_pd), verbose=0).mean()
```

np.float32(0.100023024)

Question

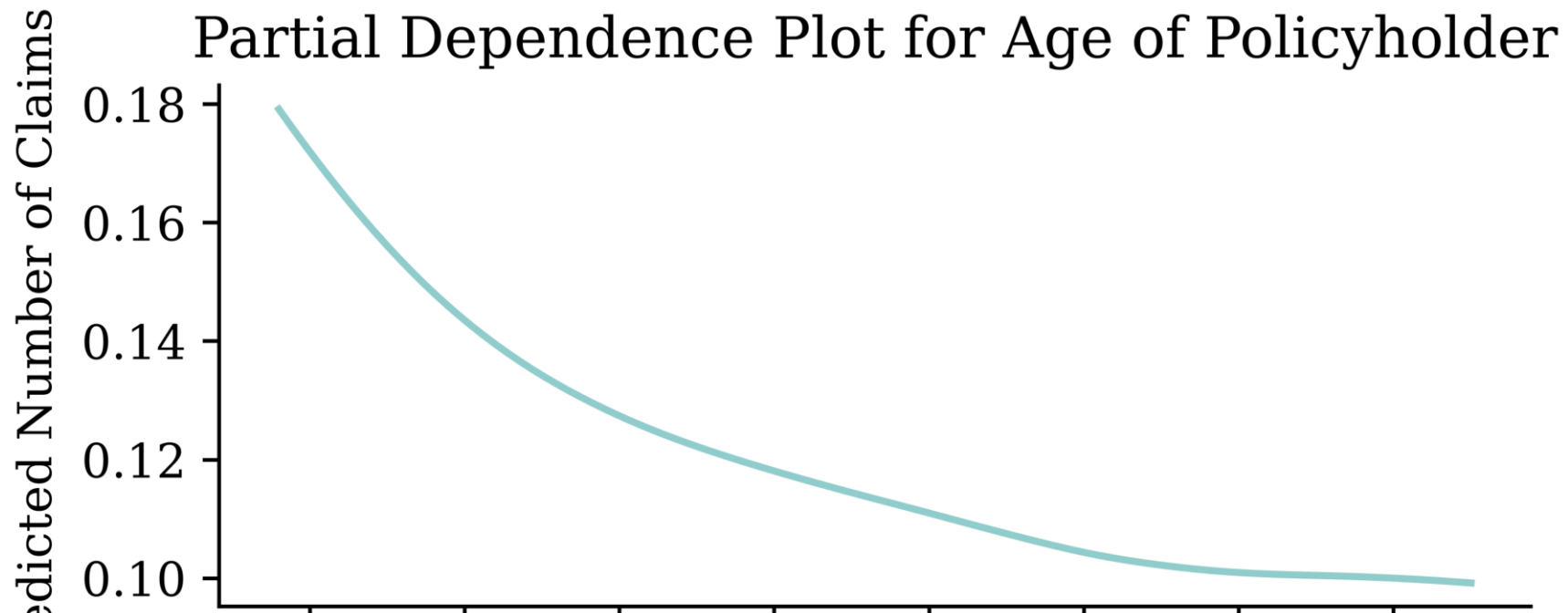
What could go wrong?

Partial dependence plots

```

1  # Make partial dependence plot for ageph by hand
2  ageph_range = np.linspace(X_train_raw['ageph'].min(), X_train_raw['ageph'].max(), 100)
3  y_preds = []
4  X_train_pd = X_train_raw.copy()
5
6  for age in ageph_range:
7      X_train_pd['ageph'] = age
8      X_train_pd_ct = ct.transform(X_train_pd)
9      y_pred_batch = nn_model.predict(X_train_pd_ct, verbose=0, batch_size=X_train_pd_ct.sh
10     y_preds.append(y_pred_batch.mean())

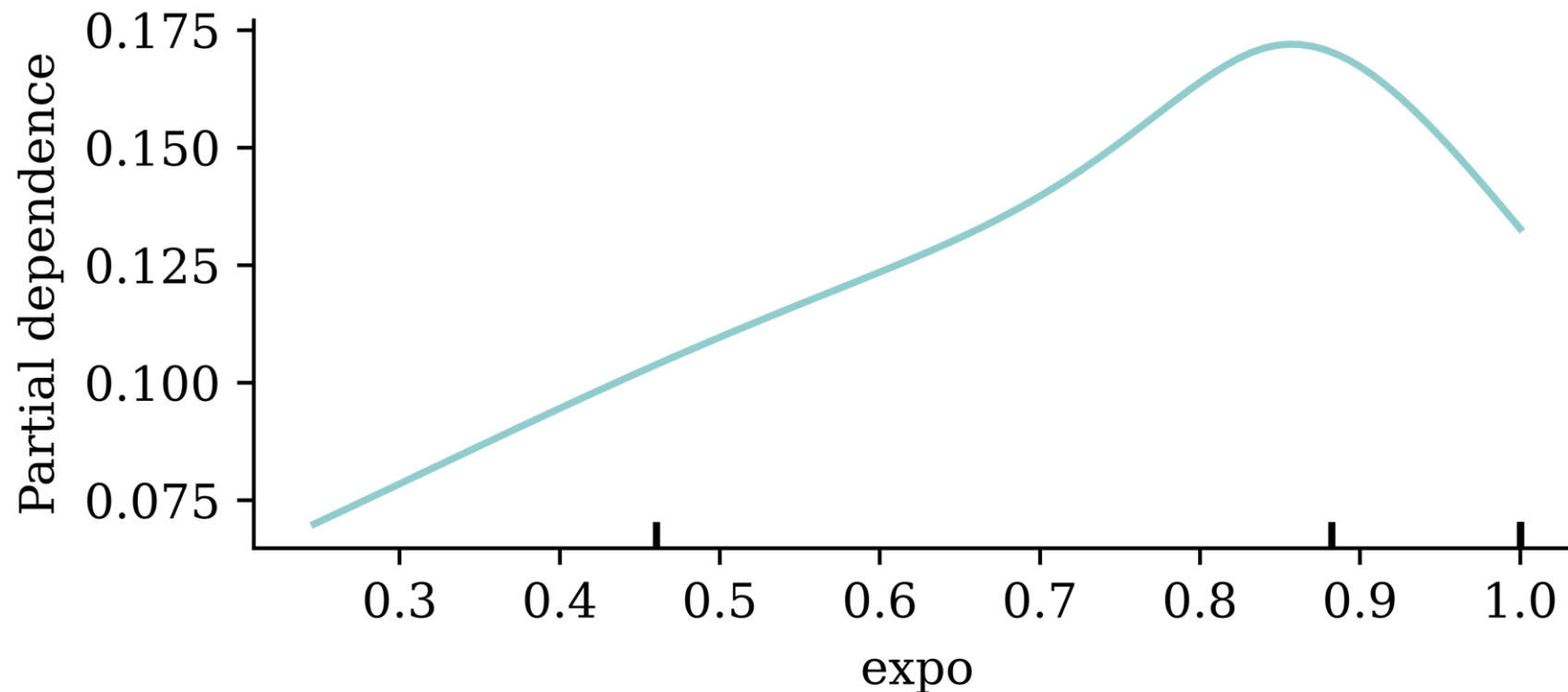
```



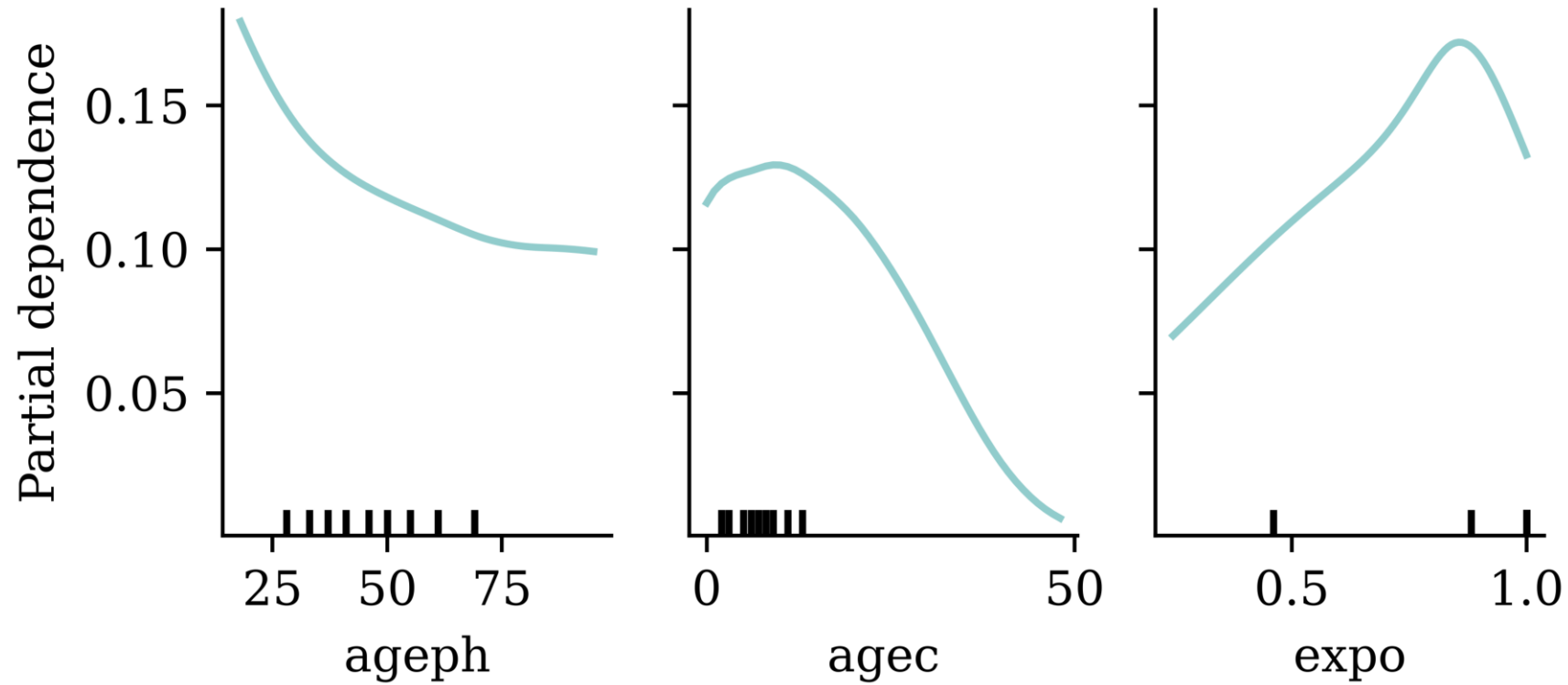
Using scikit-learn's PDP

(First need to trick scikit-learn a little..)

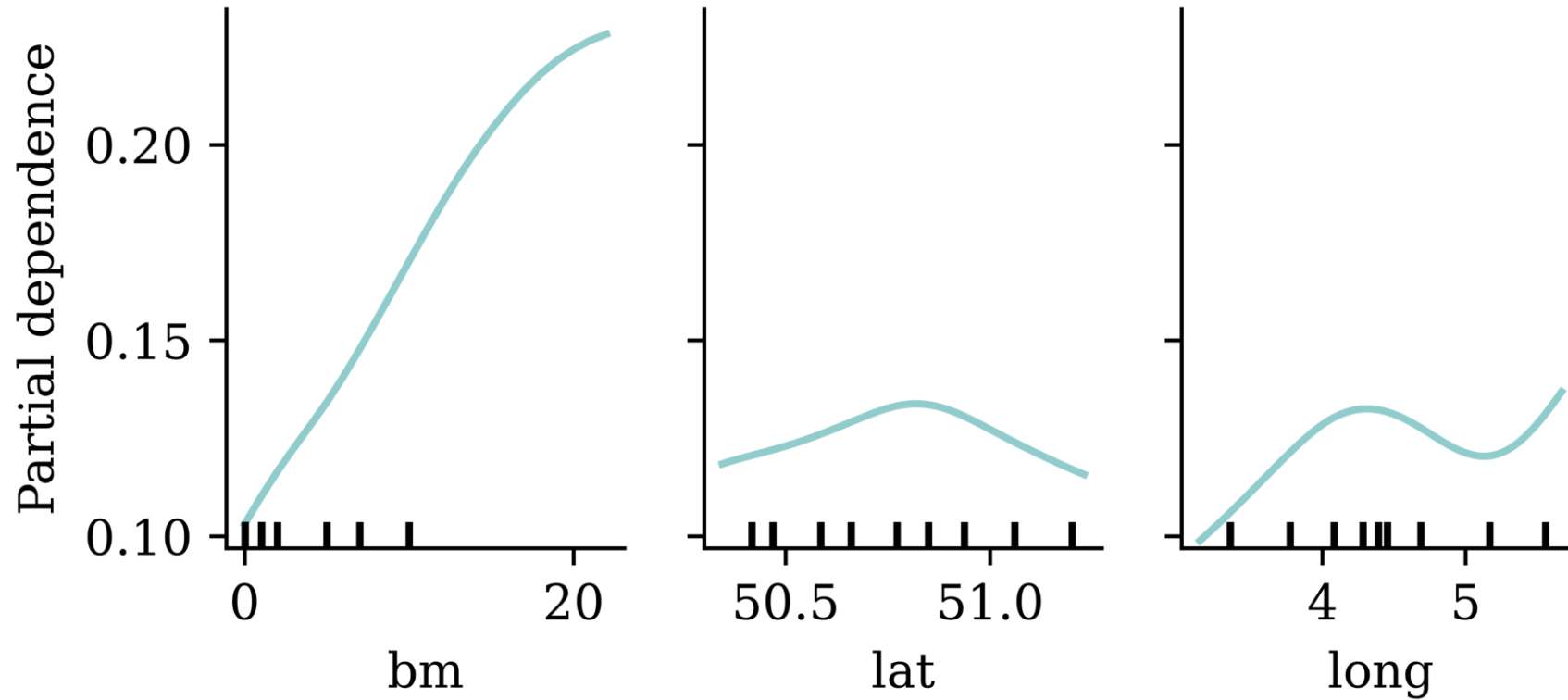
```
1 PartialDependenceDisplay.from_estimator(  
2     nn_model_skl,  
3     X_train_raw,  
4     features=[0],  
5     feature_names=X_train_raw.columns,  
6 )
```



Explaining the ages and exposure



Explaining the bonus-malus and location

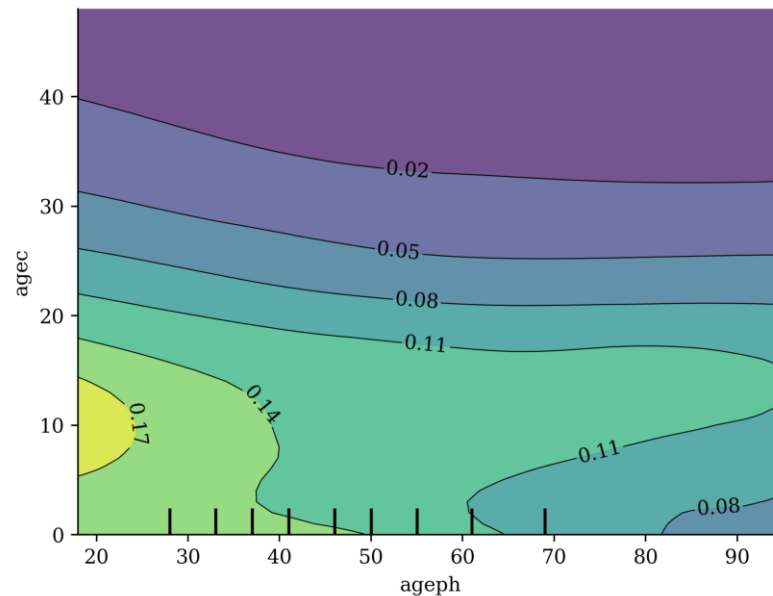


Bivariate age effects

```

1 cols = X_train_raw.columns.to_list()
2 inds = [cols.index(col) for col in ["ageph", "agec"]]
3
4 disp = PartialDependenceDisplay.from_estimator(
5     nn_model_skl,
6     X_train_raw,
7     features=[tuple(inds)],
8     feature_names=X_train_raw.columns,
9 )

```



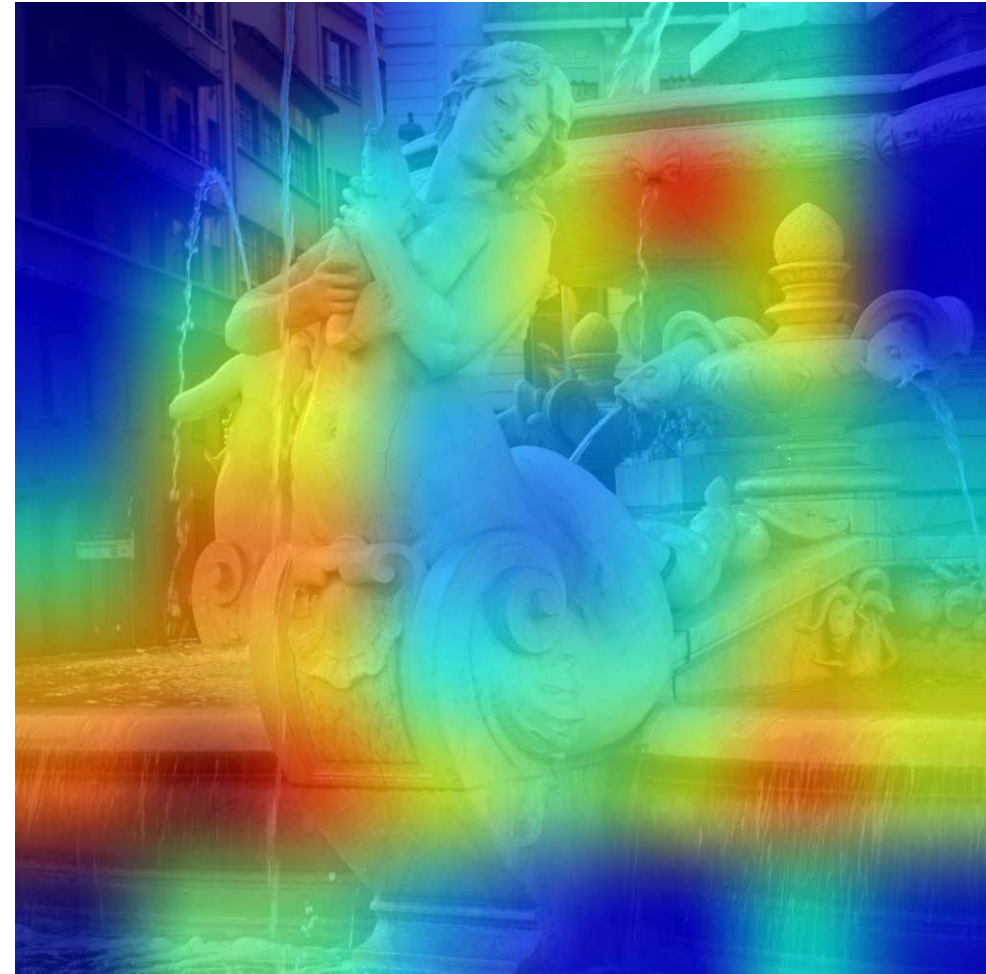
Lecture Outline

- Introduction
- Illustrative Example
- Inherently Interpretable Models
- Post-hoc Explanations
- Belgian Motor Dataset
- Interpreting Inherently Interpretable Models
- Neural Additive Models on the Belgian Data
- Explaining a Neural Network with Partial Dependence Plots
- **Explaining Specific Models**

Grad-CAM



Original image



Grad-CAM

See, e.g., [Keras tutorial](#). See Chollet (2021), Deep Learning with Python, Section 9.4.3.

Criticism

“Rather than trying to create models that are inherently interpretable, there has been a recent explosion of work on ‘explainable ML’, where a second (post hoc) model is created to explain the first black box model. This is problematic. Explanations are often not reliable, and can be misleading, as we discuss below. If we instead use models that are inherently interpretable, they provide their own explanations, which are faithful to what the model actually computes.” (Rudin, 2019)

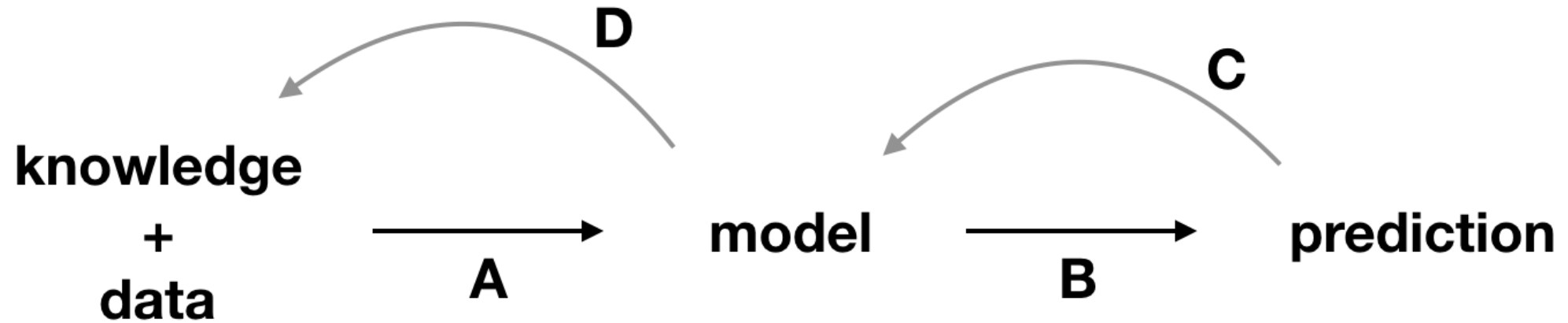
Criticism II

	Test Image	Evidence for Animal Being a Siberian Husky	Evidence for Animal Being a Transverse Flute
Explanations Using Attention Maps			

Figure 2: Saliency does not explain anything except where the network is looking. We have no idea why this image is labeled as either a dog or a musical instrument when considering only saliency. The explanations look essentially the same for both classes. Figure credit: Chaofan Chen and [28].

Multiple conflicting explanations (Rudin, 2019)

Conclusion



“Figure 20.2: Explainability techniques allow strengthening the feedback extracted from a model. A, data and domain knowledge allow building the model. B, predictions are obtained from the model. C, by analyzing the predictions, we learn more about the model. D, better understanding of the model allows better understanding of the data and, sometimes, broadens domain knowledge.” Biecek & Burzykowski (2021)

Package Versions

```
1 from watermark import watermark
2 print(watermark(python=True, packages="contextily,geopandas,keras,matplotlib,numpy,pandas
```

Python implementation: CPython

Python version : 3.14.5

IPython version : 9.15.0

contextily : 1.7.0

geopandas : 1.1.4

keras : 3.15.0

matplotlib : 3.11.0

numpy : 2.5.1

pandas : 3.0.3

plotly : 6.8.0

pygam : 0.12.0

seaborn : 0.13.2

scipy : 1.18.0

shapely : 2.1.2

sklearn : 1.9.0

Glossary

- adversarial examples
- ceteris paribus plot
- global interpretability
- Grad-CAM
- individual conditional expectation (ICE) plot
- inherent interpretability
- LIME
- local interpretability
- partial dependence plot
- permutation importance
- post-hoc explainability
- SHAP values

References

- Ben-Zion, Z., Witte, K., Jagadish, A. K., Duek, O., Harpaz-Rotem, I., Khorsandian, M.-C., Burrer, A., Seifritz, E., Homan, P., Schulz, E., et al. (2025). Assessing and alleviating state anxiety in large language models. *Npj Digital Medicine*, 8(1), 132.
- Biecek, P., & Burzykowski, T. (2021). *Explanatory Model Analysis*. Chapman; Hall/CRC, New York.
<https://pbiecek.github.io/ema/>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Carreira-Perpinán, M. A., & Tavallali, P. (2018). Alternating optimization of decision trees, with application to learning sparse oblique trees. *Advances in Neural Information Processing Systems*, 31.
- Charpentier, A. (2024). *Insurance, biases, discrimination and fairness*. Springer.
- Chen, Z., Lu, Y., Zhang, J., & Zhu, W. (2024). Managing weather risk with a neural network-based index insurance. *Management Science*, 70(7), 4306–4327.
- Delcaillau, D., Ly, A., Papp, A., & Vermet, F. (2022). Model transparency and interpretability: Survey and application to the insurance industry. *European Actuarial Journal*, 12(2), 443–484.
- Doshi-Velez, F., & Kim, B. (2017). *Towards a rigorous science of interpretable machine learning*.
- Fisher, A., Rudin, C., & Dominici, F. (2019). All models are wrong, but many are useful: Learning a variable's importance by studying an entire class of prediction models simultaneously. *Journal of Machine Learning Research*, 20(177), 1–81.
- Goodfellow, I. J., Shlens, J., & Szegedy, C. (2014). Explaining and harnessing adversarial examples. *arXiv Preprint arXiv:1412.6572*.
- Laub, P. J., Pho, T., & Wong, B. (2026). An interpretable deep learning model for general insurance pricing. *Insurance: Mathematics and Economics*, 103270.
- Li, C., Wang, J., Zhang, Y., Zhu, K., Hou, W., Lian, J., Luo, F., Yang, Q., & Xie, X. (2023). Large language models understand and can be enhanced by emotional stimuli. *arXiv Preprint arXiv:2307.11760*.
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.
- Molnar, C. (2020). *Interpretable machine learning*.
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). Why Should I Trust You?": Explaining the Predictions of Any Classifier.